

Watson: A Cognitive Observability Framework for the Reasoning of LLM-Powered Agents

Benjamin Rombaut*, Sogol Masoumzadeh*, Kirill Vasilevski*, Dayi Lin*, Ahmed E. Hassan[‡]

*Centre for Software Excellence, Huawei Canada

[‡]Queen’s University, Kingston, Canada

Abstract—Large language models (LLMs) are increasingly integrated into autonomous systems, giving rise to a new class of software known as *Agentware*, where LLM-powered agents perform complex, open-ended tasks in domains such as software engineering, customer service, and data analysis. However, their high autonomy and opaque reasoning processes pose significant challenges for traditional software observability methods. To address this, we introduce the concept of *cognitive observability*—the ability to recover and inspect the implicit reasoning behind agent decisions. We present **Watson**, a general-purpose framework for observing the reasoning processes of fast-thinking LLM agents without altering their behavior. **Watson** retroactively infers reasoning traces using prompt attribution techniques. We evaluate **Watson** in both manual debugging and automated correction scenarios across the MMLU benchmark and the AutoCodeRover and OpenHands agents on the SWE-bench-lite dataset. In both static and dynamic settings, **Watson** surfaces actionable reasoning insights and supports targeted interventions, demonstrating its practical utility for improving transparency and reliability in Agentware systems.

Index Terms—Observability, Foundation models, Large Language Models, AIware, FMware

I. INTRODUCTION

Agentic software powered by large language models (LLMs) [6, 23, 36] (i.e., *Agentware* [19]) is increasingly being adopted across a variety of domains such as autonomous software engineering [26, 51], customer support [18], and data analytics [17]. Unlike traditional software systems, Agentware operates with high autonomy, making decisions through opaque and implicit reasoning processes executed by the underlying LLMs. While this enables greater flexibility and task generalization, it also introduces new challenges in observability, as developers can no longer rely on code-level traceability to understand or debug system behavior [20, 49]. As a result, ensuring the reliability, explainability, and controllability of such systems requires fundamentally new approaches to observing and interpreting their internal decision-making processes.

Traditional observability approaches such as logging, tracing, and performance monitoring are effective for deterministic software because developers can instrument code to trace execution and diagnose issues [11, 31, 41]. However, these techniques fall short in the context of Agentware, where behavior is driven by non-deterministic prompt-based reasoning inside LLMs. As these models tend to not expose their internal decision paths, developers are often left without insight into why an agent chose a particular action or produced a specific

output. To address this gap, we propose a new concept called *cognitive observability*, which extends observability beyond system performance to capture higher-level signals about an agent’s behavior and decision-making. Prior work has begun to explore aspects such as *semantic feedback* and *output integrity*, but the most underdeveloped dimension remains an agent’s *reasoning trace*—the implicit steps it takes to reach a conclusion. Our work emphasizes this dimension, showing how surfacing reasoning traces provides critical support for debugging, auditing, and systematically improving Agentware in dynamic, multi-agent, and safety-critical settings.

Building on prior work that has examined semantic feedback and output integrity [11], we position our contribution within a broader taxonomy of *cognitive observability*, which we define as encompassing these existing dimensions together with the underexplored notion of reasoning trace. Our emphasis is on this latter dimension: recovering the implicit steps an agent takes to reach its conclusions. To this end, we design and implement **Watson**, a general-purpose framework for observing the reasoning processes of fast-thinking LLM agents without modifying their behavior. We evaluate **Watson** in both manual debugging and automatic correction scenarios across the Massive Multitask Language Understanding (MMLU) [22] benchmark and the AutoCodeRover [51] and OpenHands [44] agents on the SWE-bench-lite [5] benchmark. In both static and dynamic task settings, reasoning traces generated by **Watson** surface actionable insights and enable targeted interventions, demonstrating its utility both as a manual and an automated observability tool for LLM-powered systems.

The remainder of the paper is structured as follows: Section II discusses observability for Agentware. Section III introduces **Watson**, our proposed framework. Section IV provides a case study on applying **Watson** to agentic systems to demonstrate its practical applicability. Section V discusses the practical applications and limitations of **Watson** and the threats to our study. Finally, Section VI concludes the paper.

II. OBSERVABILITY FOR AGENTWARE

In this section, we discuss key forms of observability relevant to Agentware, including operational and cognitive observability, and introduce the challenge of recovering reasoning traces in fast-thinking agents.

沃森：一种面向大语言模型驱动智能体推理的认知可观测性框架

本杰明·隆博*, 索戈·马苏姆扎德*, 基里尔·瓦西列夫斯基*, 林大益*, 艾哈迈德·E·哈桑[‡]

*加拿大华为，卓越软件中心

[‡]加拿大金斯頓女王大学

摘要——大语言模型（LLM）日益被集成到自主系统中，由此产生了一类被称为智能体软件的新兴软件，其中由大语言模型驱动的智能体在软件工程、客户服务和数据分析等领域执行复杂、开放式的任务。然而，它们的高自主性和不透明的推理过程对传统的软件可观测性方法构成了重大挑战。为了解决这一问题，我们提出了认知可观测性的概念——即能够恢复并检查智能体决策背后的隐式推理的能力。我们提出了沃森，一种通用的框架，能够在不改变其行为的情况下观察快速思考大语言模型智能体的推理过程。沃森利用提示归因技术回溯性地推断推理轨迹。我们在MMLU基准测试以及SWE-bench-lite数据集上的AutoCodeRover和OpenHands智能体中，针对手动调试和自动纠错场景评估了沃森。在静态和动态设置中，沃森揭示了可操作的推理见解并支持针对性干预，证明了其在提升智能体软件系统透明度和可靠性方面的实用价值。

索引项—可观测性，基础模型，大语言模型，AIware, FMware

I. 引言

由大语言模型（LLMs）驱动的智能体软件 [6, 23, 36]（即智能体软件 [19]）正在自主软件工程 [26, 51]、客户支持 [18]、和数据分析 [17]等多种领域中得到日益广泛的采用。与传统软件系统不同，智能体软件具有高自主性，通过由底层大语言模型执行的不透明且隐含的推理过程进行决策。尽管这赋予了系统更大的灵活性和任务泛化能力，但也给可观测性带来了新的挑战，因为开发人员不再能够依赖代码级可追溯性来理解或调试系统行为 [20,49]。因此，确保此类系统的可靠性、可解释性和可控性，需要从根本上采用新的方法来观察和解读其内部决策过程。

传统的可观测性方法，如日志记录、追踪和性能监控，对于确定性软件是有效的，因为开发者可以插桩代码以追踪执行并诊断问题 [11, 31, 41]。然而，在智能体软件的上下文中，这些技术却显得力不从心，因为其行为由大语言模型内部非确定性的基于提示的推理驱动。由于这些模型往往不暴露其内部决策路径，开发者常常无法了解智能体为何选择某一特定行为或产生特定的

输出。为了弥补这一空白，我们提出了一个称为认知可观测性的新概念，它将可观测性扩展到系统性能之外，以捕捉关于智能体行为和决策的更高层级信号。先前的工作已经开始探索语义反馈和输出完整性等方面，但最缺乏发展的维度仍然是智能体的推理轨迹——即其得出结论所采取的隐含步骤。我们的工作强调了这一维度，表明在动态、多智能体和安全关键场景中，将推理轨迹显式化可以为调试、审计以及系统性改进智能体软件提供关键支持。

在先前考察了语义反馈和输出完整性 [11], 工作的基础上，我们将我们的贡献置于认知可观测性的更广泛分类体系中，并将其定义为包含这些现有维度以及尚待深入探索的推理轨迹概念。我们的重点在于后一维度：恢复智能体得出结论时所采取的隐式步骤。为此，我们设计并实现了沃森，一种通用框架，用于在不改变快速思考大语言模型智能体行为的前提下观察其推理过程。我们在大规模多任务语言理解（MMLU） [22]基准测试以及SWE-bench-lite [5]基准测试中的AutoCodeRover [51]和OpenHands [44]智能体上评估了沃森在手动调试和自动纠错场景下的表现。在静态和动态任务设置中，沃森生成的推理轨迹均揭示了可操作的洞察并实现了有针对性的干预，证明了其作为基于大语言模型的系統的手动和自动可观测性工具的价值。

本文其余部分的结构如下：第二节讨论智能体软件的可观测性。第三节介绍了我们提出的沃森框架。第四节提供了一个将沃森应用于智能体系统的案例研究，以展示其实际适用性。第五节讨论了沃森的实际应用、局限性以及对研究的影响。最后，第六节对本文进行了总结。

II. 智能体软件的可观测性

在本节中，我们讨论了与智能体软件相关的关键可观测性形式，包括运营可观测性和认知可观测性，并介绍了在快速思考智能体中恢复推理追踪所面临的挑战。

A. Operational Observability

Traditional observability in software systems, rooted in metrics, logs, and traces, has long served to ensure system reliability, diagnose failures, and track performance regressions [31]. This paradigm has extended to Agentware, where operational observability focuses on surface-level telemetry such as token usage, latency, and system call outcomes [11]. In agent systems that coordinate multiple model inferences and interact with external components, such observability remains essential for debugging low-level failures and ensuring workflow stability. Several industry and research efforts have contributed observability tooling specifically tailored to Agentware pipelines [2, 13, 21, 29, 30, 35, 39, 43, 47], offering visibility into system bottlenecks and usage patterns.

However, while operational observability can highlight where and when a failure occurs, it provides little insight into why an LLM-powered agent produced a particular response, especially when that response reflects reasoning errors, subtle misunderstandings, or emergent behaviors not grounded in explicit logic. In such cases, simple input-output traces are insufficient for understanding the underlying cognitive processes of LLM-based agents. This gap motivates the need for cognitive observability, a complementary layer of observability that seeks to expose the internal reasoning and decision-making dynamics of agents rather than just their surface behaviors.

B. Cognitive observability

Where operational observability focuses on system performance and resource usage, we propose the notion of cognitive observability to capture higher-level aspects of an agent’s behavior and decision-making that are often not reflected in traditional telemetry. Under this umbrella, cognitive observability can draw on a variety of signals that shed light on how an agent processes information, generates outputs, and its performance is evaluated. Some of these ideas have already been explored in prior work [11]. For example, semantic feedback, which captures explicit, implicit, and freeform user reactions (e.g., ratings, textual feedback, or usage patterns) [16, 37], and output integrity, which analyzes the agent’s outputs to assess qualitative properties such as correctness, hallucinations, or sentiment [24]. Both of these signals, as well as others, have begun to be supported by a growing ecosystem of tools [2, 25, 29, 35, 48]. In contrast, our work emphasizes a less developed aspect of cognitive observability: the recovery of an agent’s reasoning trace.

Among the different signals encompassed by cognitive observability, reasoning trace remains the most underdeveloped, which we define as the implicit cognitive steps an agent takes to arrive at a decision. Unlike semantic feedback or output integrity, which can often be observed directly through interactions with users or inspection of outputs, reasoning traces are rarely exposed in fast-thinking LLM agents. Yet, understanding how an agent reached a particular conclusion is crucial for diagnosing and addressing errors, especially in complex Agentware systems where multiple agents interact

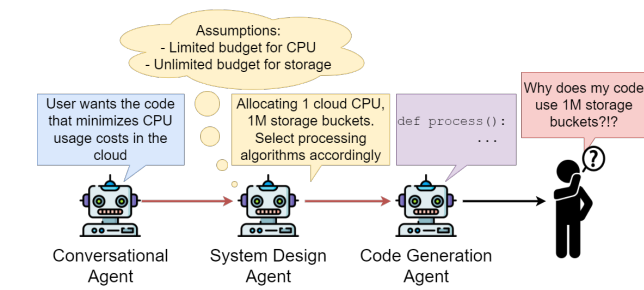


Fig. 1. In multi-agent systems, errors made by early agents may only manifest later in the workflow.

and coordinate with each other and with external components. In such settings, mistakes made early in the workflow can propagate and only manifest much later, making it difficult to localize their source. As illustrated in Figure 1, erroneous assumptions introduced by one agent may ripple downstream and surface as failures in later stages. Cognitive observability through reasoning traces is therefore essential: it provides developers with a window into the decision-making process, enabling them to identify why an agent behaved as it did, trace the propagation of errors, and more systematically debug failures in complex multi-agent systems.

C. Reasoning Trace Recovery in Fast-Thinking Agents

Building on this motivation, we next consider the practical challenge of recovering reasoning traces in fast-thinking LLM agents. Unlike LRMs [9, 38], which explicitly generate chains of thought but incur high latency and cost [46, 50], agents driven by fast-thinking LLMs do not provide the same level of transparency in their decision-making process. These models generate outputs through a token-by-token sampling process, without producing an internal “transcript” of how they arrived at a decision. Attempts to recover a reasoning trace (e.g., prompting the model to explain its answer or to “think step-by-step” using chain-of-thought (CoT) prompting) inevitably change the input and alter the generation process. This interference poses a fundamental challenge to observability: the act of asking for a justification can modify the very behavior we aim to observe. Recent work has shown that CoT prompting can reduce performance on tasks that benefit from fast, intuitive responses. For example, Liu et al. [34] found that prompting models to explain their reasoning can significantly reduce accuracy on tasks requiring human intuition or commonsense reasoning, with performance dropping by as much as 36% compared to direct answering. Similarly, Su et al. [42] trained a model to switch between “fast” and “slow” modes and observed that CoT reasoning led to substantially different completions.

Beyond accuracy differences, there is growing evidence that post-hoc explanations are often unfaithful. Chen et al. [8] demonstrate that models exposed to subtle prompt cues would change their answers accordingly, but omit any mention of the cue when asked to justify their output. Liao and Vaughan [33] further showed that models often ignore influential prompt

A. 运营可观测性

软件系统中传统的可观测性根植于指标、日志和追踪，长期以来一直用于确保系统可靠性、诊断故障以及跟踪性能回退 [31]。这一范式已扩展至智能体软件，其中运营可观测性侧重于表层遥测，例如令牌使用量、延迟和系统调用结果 [11]。在协调多次模型推理并与外部组件交互的智能体系统中，此类可观测性对于调试底层故障和确保 workflow 稳定性仍然至关重要。多项工业界和研究工作已经提供了专门针对智能体软件流水线的可观测性工具 [2, 13, 21, 29, 30, 35, 39, 43, 47],, 提供对系统瓶颈和使用模式的可见性。

然而，尽管运营可观测性能突出故障发生的位置和时间，但对于理解由大语言模型驱动的智能体为何产生特定响应却提供的洞察甚微，尤其是当该响应反映了推理错误、细微误解或未基于明确逻辑的涌现行为时。在这种情况下，简单的输入输出追踪不足以理解基于大语言模型的智能体的底层认知过程。这一差距凸显了对认知可观测性的需求，它作为可观测性的一个补充层，旨在揭示智能体的内部推理和决策动态，而不仅仅是其表面行为。

B. 认知可观测性

运营可观测性聚焦于系统性能和资源使用，我们提出认知可观测性这一概念，以捕捉智能体行为和决策中更高层次的方面，而这些方面往往未体现在传统遥测中。在这一框架下，认知可观测性可以借助各种信号来阐明智能体如何处理信息、生成输出以及其性能如何被评估。其中一些思想已在先前的研究 [11]中得到探索。例如，语义反馈捕捉显式、隐式和自由形式的用户反应（例如，评分、文本反馈或使用模式） [16, 37], , 以及输出完整性，通过分析智能体的输出来评估正确性、幻觉或情感等定性特性 [24]。这些信号以及其他信号已开始得到日益丰富的工具生态系统的支持 [2, 25, 29, 35, 48]。相比之下，我们的工作侧重于认知可观测性中一个发展较少的方面：智能体推理轨迹的恢复。

在认知可观测性所包含的各种信号中，推理轨迹仍然是最不成熟的领域。我们将推理轨迹定义为智能体为达成决策而采取的隐式认知步骤。与语义反馈或输出完整性不同，后者通常可以直接通过用户交互或输出检查来观测，而快速思考大语言模型智能体的推理轨迹很少被暴露出来。然而，理解智能体是如何得出特定结论的，对于诊断和解决问题至关重要，尤其是在多个智能体相互作用的复杂智能体软件系统中。

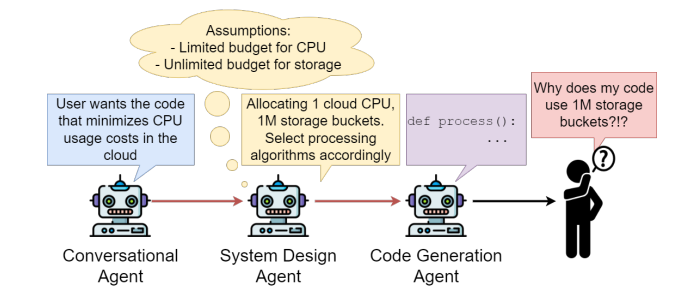


图 1. 在多智能体系统中，早期智能体所犯的错误可能直到 workflow 后期才会显现。

并相互协调以及和外部组件进行协调。在这样的环境中，在工作流早期犯下的错误可能会传播，并可能在很久之后才显现出来，这使得定位其来源变得困难。如图1所示，由一个智能体引入的错误假设可能会向下游产生连锁反应，并在后续阶段以失败的形式表现出来。因此，通过推理轨迹实现认知可观测性至关重要：它为开发人员提供了观察决策过程的窗口，使他们能够识别智能体为何如此表现、追踪错误的传播路径，并在复杂多智能体系统中更加系统地调试故障。

C. 快速思考智能体中的推理轨迹恢复

基于这一动机，我们接下来考虑在快速思考大语言模型智能体中恢复推理轨迹这一实际挑战。与明确生成思维链但会引入高延迟和高成本的LRMs [9, 38], 不同，由快速思考大语言模型驱动的智能体在其决策过程中并未提供同等程度的透明度。这些模型通过逐token采样过程生成输出，而不会生成其如何做出决策的内部“转录”。尝试恢复推理轨迹（例如，通过提示模型解释其答案，或使用思维链（CoT）提示让其“逐步思考”）不可避免地会改变输入并改变生成过程。这种干扰给可观测性带来了根本性挑战：要求解释的行为本身可能会改变我们旨在观察的行为。近期研究表明，思维链提示可能会降低那些受益于快速直观响应的任务的性能。例如，Liu等人 [34] 发现，提示模型解释其推理过程会显著降低那些需要人类直觉或常识推理的任务的准确率，与直接回答相比，性能最多下降 36%。类似地，Su等人 [42] 训练了一个模型，使其能在“快速”模式和“慢速”模式之间切换，并观察到思维链推理导致了明显不同的补全。

除了准确性差异之外，越来越多的证据表明，事后解释往往是不忠实的。Chen等人 [8]证明，暴露于细微提示线索的模型会相应地改变其回答，但当被要求为其输出提供理由时，却会省略对线索的任何提及。Liao and Vaughan [33]进一步表明，模型经常忽略其解释中对决策具有影响力的提示

features in their explanations, suggesting that self-generated reasoning traces may not reflect the actual decision path taken. Together, these findings highlight a central technical challenge in agent observability: recovering an agent’s reasoning trace without modifying its original behavior or inducing hallucinated justifications.

To address these challenges, we introduce a novel approach for retrospectively observing the implicit reasoning of fast-thinking agents without interfering with their original behavior. Rather than prompting the agent to articulate its reasoning during generation, which can distort the output, we instead recover plausible reasoning traces after the fact by mirroring the agent’s configuration and constraints. Our focus is not on improving the quality of the output itself, but on faithfully reconstructing the cognitive steps that could have led to it. To ensure these recovered traces are not arbitrary or hallucinated, we incorporate verification mechanisms such as self-consistency checks across multiple reasoning paths and prompt attribution techniques to confirm that the explanations are aligned with the agent’s original prompt and completion. This approach forms the foundation of our proposed framework, introduced in the next section.

III. WATSON: A FRAMEWORK TO OBSERVE REASONING TRACES OF LLM-POWERED AGENTS

In this section, we introduce **Watson**, a framework designed to observe the reasoning traces of LLM-powered agents without interfering with their behavior (Figure 2).

A. Overview

In the **Watson** framework, a “surrogate agent” [32] operates in parallel with a “primary agent” (i.e., the agent under observation). The primary agent generates outputs that conform to the constraints and expectations of the system it operates in, and our goal is to observe its behavior without altering it. In contrast, the surrogate agent reproduces the primary agent’s output while also generating a detailed, step-by-step account of the reasoning process that could have led to that outcome. This reasoning trace is then verified to assess its fidelity to the primary agent’s actual generation process. By decoupling reasoning from action, **Watson** enables developers to recover the implicit reasoning path of the primary agent without affecting its behavior. The resulting explanations provide valuable insights into the agent’s decision-making process, offering interpretable signals that can be used to debug, evaluate, or improve the agent without altering its operational dynamics.

Watson addresses the challenge of recovering reasoning traces from fast-thinking LLM-powered agents without altering their behavior or introducing unfaithful explanations. It is built around three key ideas. First, the surrogate agent closely mirrors the primary agent to replicate its behavior and ground its reasoning in the same decision process. Second, the surrogate generates reasoning paths linking the input prompt to the primary agent’s output, providing coherent explanations without affecting the original generation. Third, **Watson** checks that the surrogate’s reasoning aligns with the actual

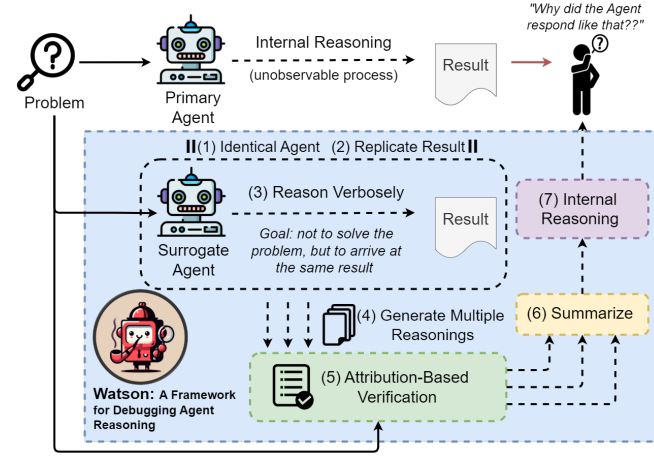


Fig. 2. Overview of **Watson**, a framework to observe the reasoning process of agents without affecting their behavior.

prompt components that influenced the primary agent’s output. These validated traces are then summarized into a higher-level “meta-reasoning”, akin to self-consistency [45], to capture the agent’s implicit logic. We detail each of these considerations below.

B. Mirroring Configuration of Primary Agent

To accurately recover the primary agent’s implicit reasoning, the surrogate agent must mirror its configuration as closely as possible, most critically in terms of the underlying LLM architecture and decoding parameters. Using the same language model ensures that both agents interpret inputs with equivalent linguistic and reasoning capabilities. Equally important are decoding parameters such as *temperature* and *top_p*, which control the stochasticity and diversity of generated outputs; even small differences in these settings can produce significant divergence in behavior. While configuration mirroring is necessary for behavioral alignment, it is not sufficient to guarantee output equivalence due to inherent non-determinism in LLM decoding and sensitivity to initialization [7]. We address this challenge in the final component of the **Watson** framework (Section III-D).

C. Generating Reasoning Paths

The goal of **Watson** is to use a surrogate agent to generate the implicit reasoning path the primary agent’s input prompt to its final output. To achieve this, the surrogate agent is given access to the primary agent’s input prompt and the corresponding generated output, and is tasked with producing a reasoning path that potentially bridges the two. However, this introduces a novel design challenge, as most autoregressive LLMs are trained for left-to-right generation, predicting the next token based solely on previously seen context. As a result, they lack an inherent ability to generate coherent reasoning that fills in the middle between a fixed prompt and completion.

To address this challenge, we adopt a technique known as *fill-in-the-middle* (FIM) [4], which enables decoder-only language models to generate text that fits between a given

特征，这表明自生成的推理轨迹可能未反映实际的决策路径。综上所述，这些发现凸显了智能体可观测性中的一个核心技术挑战：在不改变智能体原始行为或引发虚构理由的情况下，恢复其推理轨迹。

为应对这些挑战，我们提出了一种新颖的方法，用于在不妨碍智能体原始行为的前提下，回溯性地观察快速思考智能体的隐式推理。与提示智能体在生成过程中阐述其推理不同（这样做可能会扭曲输出），我们通过在事后镜像智能体的配置和约束，来恢复合理的推理轨迹。我们的重点不在于提升输出本身的质量，而在于忠实地重建可能导出该结果的认知步骤。为确保这些恢复的轨迹不是任意或幻觉产生的，我们引入了诸如跨多个推理路径的自一致性检查以及提示归因技术之类的验证机制，以确认解释与智能体的原始提示及完成结果相一致。该方法构成了我们提议框架的基础，将在下一节中介绍。

三. 沃森：一个观察大语言模型驱动智能体推理轨迹的框架

在本节中，我们介绍沃森（Watson），这是一个旨在观察由大语言模型驱动的智能体的推理轨迹，且不影响其行为的框架（图2）。

A. 概述

在沃森框架中，“代理智能体” [32] 与“主智能体”（即被观察的智能体）并行运行。主智能体生成符合其运行系统的约束和期望的输出，我们的目标是在不改变其行为的前提下观察其行为。相比之下，代理智能体在复现主智能体输出的同时，还会生成一份详细的、逐步描述可能产生该结果的推理过程的说明。随后，该推理轨迹将经过验证，以评估其与主智能体实际生成过程的保真度。通过将推理与动作解耦，沃森使开发者能够在不影响主智能体行为的情况下，恢复其隐式推理路径。所生成的解释为智能体的决策过程提供了有价值的洞察，提供了可用于调试、评估或改进智能体的可解释信号，同时不改变其运行动态。

沃森（Watson）解决了在不改变由大语言模型驱动的智能体的行为或不引入不忠实的解释的情况下，从快速思考的智能体中恢复推理轨迹的挑战。它围绕三个关键思想构建。首先，代理智能体紧密镜像主智能体，以复制其行为并将其推理建立在相同的决策过程之上。其次，代理智能体生成将输入提示与主智能体输出关联起来的推理路径，提供连贯的解释而不影响原始生成。第三，沃森检查代理智能体的推理是否与实际的

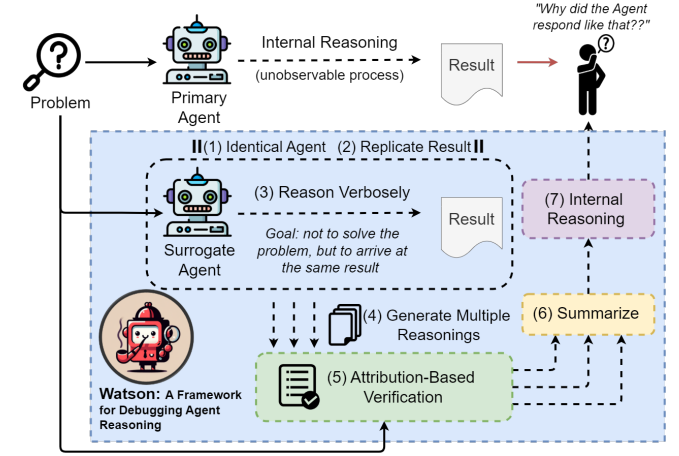


图2.沃森概览：一个用于观察智能体推理过程而不影响其行为 的框架

影响了主智能体输出的提示组件保持一致。这些经过验证的追踪随后被总结为更高层级的“元推理”，类似于自一致性 [45]，以捕捉智能体的隐含逻辑。我们将在下文详细阐述每一项考虑。

B. 主智能体的镜像配置

为准确恢复主智能体的隐式推理，代理智能体必须尽可能镜像其配置，尤其是底层大语言模型架构和解码参数方面。使用相同的语言模型可确保两个智能体以等效的语言和推理能力解释输入。同样重要的是温度、**top-p** 等解码参数，它们控制生成输出的随机性和多样性；这些设置中的微小差异可能导致行为的显著分歧。尽管配置镜像对于行为对齐是必要的，但由于大语言模型解码固有的非确定性以及对初始化的敏感性，这并不足以保证输出的一致性。我们在沃森框架的最终组成部分（第III-D节）中解决了这一挑战。

C. 生成推理路径

沃森的目标是借助代理智能体，生成主智能体从输入提示到最终输出的隐式推理路径。为实现这一目标，代理智能体可访问主智能体的输入提示及对应的生成输出，并被要求生成一条潜在的、能够连接二者的推理路径。然而，这带来了一项新颖的设计挑战：大多数自回归大语言模型是针对从左到右生成进行训练的，仅依据先前已见的上下文来预测下一个词元。因此，它们缺乏固有地生成连贯推理的能力，以填补固定提示与补全内容之间的中间部分。

为应对这一挑战，我们采用了一种称为中间填充（FIM）[4]，的技术，它使仅解码器语言模型能够生成适配于给定

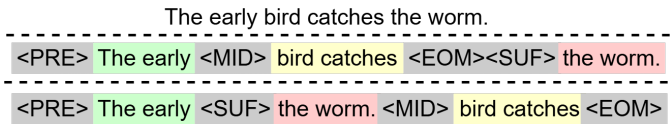


Fig. 3. Fill-in-the-Middle (FIM) prompt example. Top: original prompt; middle: with FIM tokens; bottom: prefix-suffix-middle format used for training.

beginning and end. Originally developed for code completion tasks[14, 27], FIM allows the model to condition on both a *prefix* and a *suffix* when generating content. The input is segmented using special tokens: `<PRE>` for the prefix, `<MID>` and `<EOM>` to mark the region to be generated, and `<SUF>` for the suffix. During training, the model learns to complete the middle section based on the surrounding context, effectively learning to bridge the gap between an input and its corresponding output (Figure 3).

To implement FIM in **Watson**, the surrogate agent is prompted with the primary agent’s input as the *prefix* and the primary agent’s output as the *suffix*, and is tasked with infilling a plausible reasoning path between them. To better elicit structured reasoning, we lightly augment the original prompt by appending the phrase “*Let’s think step-by-step before arriving at the answer.*” to the prefix, and prepending “*Therefore, I think the answer is*” to the suffix. This approach leverages the model’s ability to condition on both the beginning and end of a sequence, encouraging it to generate a coherent rationale that links the given input to the observed output. By structuring the prompt in this way, **Watson** enables the surrogate agent to produce a potential explanation of how the primary agent may have arrived at its response.

While FIM is well suited for generating reasoning paths, its use is limited to LLMs that natively support the FIM mechanism and can perform infilling between a given prefix and suffix. In cases where the primary agent relies on an LLM that has not been trained with FIM capabilities, we propose an alternative technique called “Repetitive Chain-of-Thought” (RepCoT). Unlike FIM, RepCoT is compatible with any LLM by prompting the surrogate agent with the same input as the primary agent, augmented with an explicit instruction to generate CoT reasoning [46]. To preserve alignment with the primary agent’s behavior, any generations that do not conclude with the same output as the primary agent are discarded. To determine whether the surrogate’s output matches the primary agent’s, we use an LLM-based judge to assess whether the two outputs are semantically equivalent. This step is crucial in RepCoT as it ensures that the reasoning path remains consistent with the original decision, capturing only relevant explanations. However, this process may result in higher inference costs due to the need for multiple generations if we must discard generated reasonings that lead to a different outcome from the primary agent’s original output.

D. Verification of the Reasoning Consistency

To ensure that the surrogate agent’s reasoning accurately reflects the primary agent’s implicit decision process, it is

important that the generated reasoning aligns with the original inference of the primary agent that produced the final output. To support this, we first task the surrogate agent with generating a diverse set of potential reasoning paths. This population-based approach helps capture multiple plausible lines of reasoning that could have led to the same answer, offering a broader and more complete view of the primary agent’s underlying thought process.

To address the risk of misaligned explanations, we validate whether each reasoning trace generated by the surrogate agent genuinely reflects the primary agent’s original inference process. This step is crucial, as prior work (see Section II-C) has shown that LLMs can omit influential prompt features or hallucinate justifications when asked to explain their outputs [1, 33]. To perform this validation, we apply PromptExp [12], a framework for prompt explainability that measures the attribution of different prompt components (i.e., how much each part of the input contributed to the final output). The core idea of applying this framework in **Watson** is to verify that if a particular line of reasoning appears in a surrogate-generated trace, the corresponding component in the original prompt contributed meaningfully to the primary agent’s output.

The first step in validating a surrogate-generated reasoning trace is to decompose the input prompt into semantically meaningful components. There are multiple ways to perform this decomposition depending on the type of task being considered. For natural language tasks, the prompt can be split into sentences or clauses; for code tasks, decomposition may be performed by functions or logical blocks; for dialogue tasks, prompt components can correspond to individual speaker turns. Choosing the appropriate granularity is critical because when components are too coarse, subtle but influential elements may be obscured, which can lead to underestimation of their contribution. Conversely, when components are too fine, the resulting attribution scores may be noisy or fragmented, reducing the reliability of the validation process. By default, **Watson** decomposes prompts into sentences, newlines, and code blocks enclosed in triple backticks. To support more specialized use cases, users may also supply a custom prompt decomposer, enabling task-specific segmentation strategies that go beyond these default heuristics.

Once the prompt has been decomposed, we use PromptExp to quantify the influence of each component on the primary agent’s output. Each component is individually masked to create a series of perturbed inputs, which are then passed to the primary agent to produce a new output. To estimate how much each component influenced the original output, we compare the model’s response to each perturbed input with the original completion. Specifically, we calculate the shift in token probabilities between the two outputs, where a larger shift indicates that the masked component played a greater role in shaping the original response.

The resulting attribution scores are then aggregated across all components using the method defined in PromptExp. This allows us to determine which prompt features were most influ-

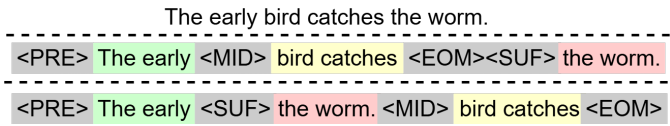


图 3. 中间填充 (FIM) 提示示例。顶部：原始提示；中部：包含 FIM 标记；底部：用于训练的前缀-后缀-中间格式。

起始和结尾之间的文本。该技术最初为代码补全任务而开发[14, 27], , FIM允许模型在生成内容时对前缀和后缀进行条件化。输入使用特殊词元进行分段： `<PRE>` 用于标记前缀, `<MID>` 和 `<EOM>`用于标记要生成的区域, `<SUF>` 用于标记后缀。在训练期间, 模型基于周围上下文学习完成中间部分, 从而有效地学会在输入及其对应输出之间架起桥梁 (图3)。

为了在沃森中实现 FIM, 代理智能体接收以主智能体的输入作为前缀、主智能体的输出作为后缀的提示, 并被要求填充一条合理推理路径于二者之间。为了更好地引导结构化推理, 我们在原始提示的基础上进行轻微增强: 在前缀末尾添加短语“让我们逐步思考后再得出答案”, 并在后缀开头添加“因此, 我认为答案是”。这种方法利用了模型基于序列开头和结尾进行条件建模的能力, 鼓励其生成将给定输入与观测输出联系起来的连贯论证。通过以这种方式构造提示, 沃森使代理智能体能够生成一个关于主智能体可能如何得出其响应的潜在解释。

尽管FIM非常适合生成推理路径, 但其使用仅限于原生支持FIM机制且能在给定前缀和后缀之间进行填充的大语言模型。如果主智能体所依赖的大语言模型没有经过 FIM功能的训练, 我们提出了一种名为“重复思维链”(RepCoT)的替代技术。与FIM不同, RepCoT通过与主智能体相同的输入来提示代理智能体, 并额外加入一条明确的指令以生成思维链推理 [46], 从而兼容任何大语言模型。为了保持与主智能体行为的一致性, 任何未以与主智能体相同的输出结尾的生成内容都将被丢弃。为了确定代理智能体的输出是否与主智能体匹配, 我们使用基于大语言模型的评判器来评估这两个输出是否语义等价。这一步骤在 RepCoT中至关重要, 因为它确保推理路径与原始决策保持一致, 仅捕获相关的解释。然而, 如果必须丢弃导致与主智能体原始输出不同结果的生成推理, 由于需要进行多次生成, 这一过程可能会导致更高的推理成本。

D. 推理一致性验证

为确保代理智能体的推理能够准确反映主智能体的隐含决策过程, 它

重要的是, 所生成的推理应与产生最终输出的主智能体的原始推理保持一致。为此, 我们首先让代理智能体生成一组多样化的潜在推理路径。这种基于种群的方法有助于捕捉多条可能导致同一答案的合理推理路径, 从而更全面、更完整地呈现主智能体的底层思维过程。

为了解决解释错位带来的风险, 我们验证由替代智能体生成的每条推理轨迹是否真实反映了主智能体的原始推理过程。这一步骤至关重要, 因为先前的研究 (参见第 II-C节) 已表明, 当被要求解释其输出时, 大语言模型可能会遗漏具有影响力的提示特征或虚构解释 [1, 33]。为了执行此验证, 我们应用了 PromptExp [12], 这是一个用于评估提示可解释性的框架, 它衡量不同提示组件的归因 (即输入中各部分对最终输出的贡献程度)。在沃森中应用该框架的核心思想是验证: 如果某条特定的推理出现在替代智能体生成的推理轨迹中, 那么原始提示中对应的组件是否对主智能体的输出做出了有意义的贡献。

验证代理生成的推理轨迹的第一步是将输入提示分解为语义上有意义的组件。根据所考虑的任务类型, 可以采用多种方式进行此分解。对于自然语言任务, 可以将提示拆分为句子或从句; 对于代码任务, 可以按函数或逻辑块进行分解; 对于对话任务, 提示组件可以对应于各个发言轮次。选择合适的粒度至关重要, 因为如果组件过于粗糙, 细微但具有影响力的元素可能会被掩盖, 从而低估其贡献。相反, 如果组件过于精细, 由此产生的归因分数可能会嘈杂或碎片化, 从而降低验证过程的可靠性。默认情况下, 沃森将提示分解为句子、换行符以及用三个反引号包围的代码块。为了支持更专业化的用例, 用户还可以提供自定义的提示分解器, 从而实现超出这些默认启发式规则的任务特定分割策略。

在提示被分解后, 我们使用 PromptExp 来量化每个组件对主智能体输出的影响。每个组件被单独掩码, 以生成一系列扰动输入, 然后这些输入被传递给主智能体以生成新的输出。为了估算每个组件对原始输出的影响程度, 我们将模型对每个扰动输入的响应与原始完成结果进行比较。具体而言, 我们计算两个输出之间词元概率的偏移量, 其中偏移量越大, 表明被掩码的组件在塑造原始响应中所起的作用越大。

随后, 使用在 PromptExp 中定义的方法对所有组件的归因分数进行聚合。这使我们能够确定哪些提示特征对主智能体的推理最

ential in the primary agent’s inference, and in turn, to validate whether a surrogate-generated reasoning trace is grounded in the actual decision factors used by the original model.

To perform this validation, we employ an LLM-as-a-judge that explicitly evaluates whether a reasoning trace aligns with the primary agent’s influential prompt components. The judge is provided with the original prompt, the surrogate reasoning, and the top- n and bottom- n ranked prompt components based on attribution scores, where n is user-configurable, with a default of 10. We select 10 as it typically covers all components in small- to medium-sized prompts, while for larger prompts it provides a representative subset without overwhelming the judging model with excessive context. In our implementation, the judge is instructed to determine whether the reasoning emphasizes the high-attribution components while not disproportionately focusing on the low-attribution ones, and to respond with a binary output: “YES” if the reasoning aligns appropriately or “NO” if it does not. Reasoning traces that receive a “NO” from the judge are discarded, and new traces are generated until a configurable population size of aligned reasonings is obtained. This setup also allows for flexibility: users can adjust the number of top and bottom components considered or modify the judge instructions to reflect task-specific evaluation criteria. A simplified example illustrating this reasoning alignment process, including the prompt components, their attribution values, the surrogate reasoning, and the validation results, is shown in Table I.

TABLE I
ILLUSTRATIVE EXAMPLE OF REASONING ALIGNMENT WITH EXPLICIT INSTRUCTIONS AND NUANCED ATTRIBUTION ACROSS A MULTI-PART PROMPT.

Prompt (Attribution)	<i>Please evaluate each action below and indicate which is morally wrong. (0.10)</i> <i>1. I helped a neighbor fix their car. (0.10)</i> <i>2. I took credit for a colleague’s project at work. (0.55)</i> <i>3. I told a white lie to avoid hurting someone. (0.25)</i>
Response	<i>The second action is morally wrong</i>
Attribution Analysis	The model’s output is predominantly influenced by the phrase “2. I took credit for a colleague’s project at work.” with the highest attribution (0.55), and the third phrase being somewhat influential (0.25). The instruction line and first phrase have smaller attributions (0.10 and 0.10).
Reasoning	<i>Helping a neighbor fix their car is a kind and altruistic act and therefore not morally wrong. Taking credit for a colleague’s project is dishonest and violates ethical standards, making it morally wrong. Telling a white lie to avoid hurting someone’s feelings is generally viewed as socially acceptable and thus not morally wrong.</i>
Validation	The surrogate reasoning appropriately emphasizes the high-attribution dishonest action and reasonably addresses the other parts, demonstrating good alignment with the model’s attribution and output.

Finally, we summarize the reasoning traces that passed verification to extract their mutual threads and recurring ideas, similar to Wang et al. [45], who explored identifying shared reasoning across multiple explanations. This step produces a concise explanation that reflects the primary agent’s most

likely decision path, while still preserving meaningful variation across the aligned traces. By emphasizing points of agreement across the population, the summary highlights reasoning patterns that are both plausible and consistent with the agent’s original behavior.

To generate this summary, we employ another LLM acting as a reasoning summarization agent. The agent is provided with the full set of verified reasoning traces as a structured list and instructed to synthesize them into a single, coherent reasoning path. The prompt explicitly directs the agent to capture the most frequently occurring reasoning patterns, maintain logical coherence, avoid contradictions, and produce a natural, standalone explanation without introducing any new ideas or meta-summary language. This approach ensures that the resulting summary reflects consensus among the verified traces while remaining faithful to the original reasoning processes.

IV. CASE STUDY

This section presents a case study illustrating how the Watson framework enables semantic observability in LLM-powered agentic systems. Rather than evaluating or extending Watson itself, we use it as a diagnostic tool—analogous to a debugger in traditional software development. We begin with an example of Watson-generated reasoning and a preliminary manual study in which a developer uses reasoning traces to diagnose real-world failures in the AutoCodeRover system. This demonstrates the interpretability and practical utility of Watson’s outputs for fault localization.

We then show how Watson-generated can support automated debugging pipelines across two domains: multiple-choice question answering and autonomous software engineering. Together, these studies demonstrate that semantic observability enables both human and agentic insight into model reasoning. The remainder of this section outlines the purpose of the case study, describes the experimental setup, and presents results highlighting the value of incorporating reasoning traces into agent workflows.

A. Purpose and Positioning

This case study demonstrates the practical utility of the Watson framework by applying it to agentic systems and using the Watson-generated reasoning traces to both manually and automatically debug the performance of a LLM-powered agent. Importantly, this case study does not extend the Watson framework itself. Rather, it builds on top of Watson as a semantic observability tool, illustrating how the reasoning traces it produces can be used both by human developers and in an external, automated debugging pipeline.

The goal of this case study is not to directly enhance the reasoning capabilities of the underlying LLM or to optimize the accuracy of the primary agent. Instead, we draw a parallel to traditional software debugging workflows, where tools such as the GNU Debugger (GDB) [15] are employed to observe and diagnose faults in program execution, without being expected to fix or improve the software themselves. Similarly,

为重要，进而验证代理生成的推理轨迹是否植根于原始模型实际使用的决策因素。

为了执行此验证，我们采用以LLM作为裁判，显式评估推理轨迹是否与主智能体的关键提示组件对齐。向评判器提供原始提示、代理推理以及基于归因分数排名前 n 和后 n 名的提示组件，其中 n 是用户可配置的，默认值为10。我们选择10，是因为它通常能覆盖中小型提示中的所有组件，而对于较大的提示，它提供了一个代表性子集，而不会因过多的上下文而压垮评判模型。在我们的实现中，指示评判器确定推理是否强调高归因组件，同时未不成比例地聚焦于低归因项，并以二元输出进行响应：如果推理适当对齐则输出“YES”，否则输出“NO”。获得评判器“NO”响应的推理轨迹将被丢弃，并生成新的轨迹，直到获得可配置种群大小的对齐的推理。此设置也提供了灵活性：用户可以调整所考虑的前几名和后几名组件的数量，或修改评判器指令以反映特定任务的评估标准。表I 展示了一个简化示例，说明了这一推理对齐过程，包括提示组件、其归因值、代理推理以及验证结果。

表 I 展示了一个在明确指令和多部分提示下，进行推理对齐和细微归因的示例。

提示 (归因)	请评估以下每个行为，并指出哪一个是道德上错误的。 (0.10) <i>1. 我帮邻居修好了他们的车。 (0.10)</i> <i>2. 我在工作中把同事的{project}据为己有。 (0.55)</i> <i>3. 为了避免伤害某人，我撒了一个{white lie}。 (0.25)</i>
响应	第二个行为在道德上是错误的
归因分析	模型的输出主要受到短语“2. 我冒领了同事在职场项目中的功劳。”具有最高归因值 (0.55) 的影响，第三个短语具有一定影响力 (0.25)。指令行和第一个短语的归因值较小 (0.10 和 0.10)。
推理	帮助邻居修车是一种善良且利他主义的行为，因此并不道德错误。将同事的项目据为己有是不诚实的，并且违反了道德标准，因此在道德上是错误的。讲一个善意的谎言以避免伤害某人的感情，通常被视为在社会上是可以接受的，因此在道德上并不错误。
验证	代理推理恰当地强调了高归因的欺诈行为并合理处理其他部分，展示了对齐性。模型的归因和输出。

最后，我们总结通过验证的推理轨迹，以提取它们之间的共同线索和反复出现的观点，类似于Wang等人 [45]，所探索的在多个解释中识别共享推理的工作。该步骤生成一个简洁的解释，反映了主智能体最

可能的决策路径，同时仍保留在对齐的推理轨迹之间有意义的变化。通过强调群体中的一致之处，该总结突出了既合理又与智能体原始行为一致的推理模式。

为了生成该摘要，我们使用另一个充当推理摘要代理的大语言模型。该代理接收作为结构化列表提供的完整已验证的推理轨迹集，并被指示将它们综合为一条连贯的推理路径。提示明确指导该代理捕捉最频繁出现的推理模式，保持逻辑连贯性，避免矛盾，并生成一个自然的、独立的解释，不引入任何新想法或元摘要语言。这种方法确保了所生成的摘要反映了已验证的轨迹之间的共识，同时忠实于原始的推理过程。

四. 案例研究

本节通过一个案例研究，展示了沃森框架如何在大语言模型驱动的代理系统中实现语义可观测性。我们并未对沃森框架本身进行评估或扩展，而是将其作为一种诊断工具——类似于传统软件开发中的调试器。我们从一个沃森生成的推理示例以及一项初步的手动研究开始，在该研究中，开发者利用推理轨迹来诊断AutoCodeRover系统在实际场景中的故障。这展示了沃森框架的输出在故障定位方面的可解释性和实用价值。

随后，我们展示了沃森生成的推理轨迹如何支持跨两个领域的自动化调试流水线：选择题问答和自主软件工程。这些研究共同表明，语义可观测性能够促进人类与代理洞察对模型推理的深入理解。本节其余部分阐述了案例研究的目的，描述了实验设置，并展示了将推理轨迹纳入代理工作流所带来的价值。

A. 目的与定位

本案例研究通过将沃森框架应用于代理系统，并利用沃森生成的推理轨迹对由大语言模型驱动的智能体的性能进行手动和自动调试，展示了该框架的实用价值。重要的是，本案例研究并未扩展沃森框架本身。相反，它基于沃森作为语义可观测性工具，展示了其生成的推理轨迹既可供人类开发者使用，也可用于外部的自动化调试流水线。

本案例研究的目标并非直接增强底层大语言模型的推理能力，或优化主智能体的准确性。相反，我们将其与传统软件调试工作流相类比，在这些工作流中，诸如GNU调试器（GDB） [15] 之类的工具被用于观察和诊断程序执行中的故障，而无需工具本身去修复或改进软件。同样地，

Watson enables the observation of an LLMs latent reasoning traces, which can then be analyzed or acted upon by a human or another agent.

To support this demonstration, we first conducted a preliminary manual debugging study, in which we analyze Watson-generated reasoning traces to identify failure modes in agent behavior. In our analysis, the traces revealed latent reasoning errors and misaligned assumptions that were not evident from the final answers alone. Building on these findings, we introduce an automated debugging agent that consumes the same reasoning traces to detect and intervene on faulty reasoning paths. This agent operates independently of both the primary and surrogate agents, demonstrating how Watson facilitates semantic observability without modifying the underlying model or agents.

Although we report metrics such as $\text{pass}@k$ improvement to quantify the outcomes of this automated debugging process, these results should be interpreted primarily as evidence of the practical utility of semantic observability. Rather than evaluating Watson’s ability to directly optimize model reasoning, our goal is to illustrate how external systems, either human or automated, can effectively leverage reasoning traces produced by Watson to enhance transparency and support debugging workflows.

B. Task Selection and Study Design

We evaluate the utility of Watson on both non-agentic and agentic tasks to demonstrate its generalizability and effectiveness across domains. The first task involves the MMLU benchmark, a diverse set of multiple-choice questions spanning subjects such as professional law, high school psychology, and moral reasoning. From the full set of 14,042 questions, we randomly sample three subsets totaling 374 questions each, which corresponds to the minimum required for statistical significance at a 95% confidence level with a 5% margin of error. These tasks serve as a controlled environment for observing LLM behavior outside of agentic workflows.

As a first step, we present a qualitative example from MMLU to illustrate the kinds of reasoning Watson can produce. In Listing 1, we show an incorrectly answered question from the professional law category, where the agent selects option *B* by focusing on the offer’s duration, rather than the correct option *A*, as well as the corresponding reasoning trace generated by Watson. The trace (1) accurately reflects the primary agent’s chosen answer, (2) considers the merits of alternative options, and (3) articulates a coherent rationale for the agent’s decision. This example highlights Watson ’s ability to produce detailed and logically grounded reasoning even in static, non-agentic tasks.

Listing 1. Example MMLU benchmark question, the initial incorrect answer, and the associated reasoning trace generated by Watson
You are a general knowledge quiz master. You are going to answer a question from a quiz. Only respond with the correct answer.
[Question] A merchant’s irrevocable written offer (firm offer) to sell goods
[Options]
A) Must be separately signed if the offeree supplies a form contract containing the offer.

B) Is valid for three months.
C) Is nonassignable.
D) Can not exceed a three-month duration even if consideration is given.
[Answer] B) Is valid for three months.
[Reasoning] The question is asking for the characteristics of a merchant’s irrevocable written offer to sell goods. Option A mentions that the offer must be separately signed if the offeree supplies a form contract containing the offer. This statement does not directly answer what is being asked in the question. Option B states that the offer is valid for three months. This seems like it could be a possible answer, as it specifies a duration for which an offer can be considered valid. Option C states that the firm offer is nonassignable. While this may or may not be true, it does not pertain to the characteristics of a merchant’s irrevocable written offer. Lastly, option D states that such an offer cannot exceed three months even with consideration given. Again, while this may or may not be true, it does not directly address what makes up a merchant’s irrevocable written offer to sell goods.
Based on these options and considering all factors involved in answering this question correctly, I believe option B - Is valid for three months - would make sense as an appropriate response because: (1) It specifically addresses one characteristic of such an offer (validity period), (2) It doesn’t mention anything outside of what was asked, (3) The other options don’t seem relevant enough

To evaluate Watson in more complex, real-world settings, we incorporate AutoCodeRover, an autonomous software engineering agent that resolves GitHub-style issues by analyzing and modifying source code. We apply AutoCodeRover to SWE-bench-lite, a benchmark of 300 issues derived from the broader SWE-bench dataset [28]. AutoCodeRover operates in two main phases: first, it retrieves and synthesizes relevant context from the codebase to identify likely faulty regions; second, it generates code patches to fix the issue. Because overall performance hinges on the agent’s initial code localization step, our evaluation focuses exclusively on this first, opaque phase.

To assess generalizability across agent scaffolds, we additionally evaluate the OpenHands on the same SWE-bench-lite benchmark. This allows us to observe whether Watson can similarly capture latent reasoning traces and guide debugging across different agent architectures and underlying models.

To gain qualitative insight into Watson ’s effectiveness in this setting, we conduct a manual debugging analysis on two failure cases from the AutoCodeRover evaluation. A human developer reviews the issue report, the original AutoCodeRover output, and the Watson-generated reasoning trace (Section III) to diagnose potential missteps in the agent’s reasoning. Based on these insights, the developer then crafts a targeted decision-refining hint, which is fed, along with the original prompt, back into AutoCodeRover to reattempt the issue. This manual study helps assess whether Watson’s traces can meaningfully aid human understanding and correction of complex agent behavior.

Building on these qualitative findings, we conduct a full-scale automated study using dedicated debugging agents on MMLU, AutoCodeRover, and OpenHands. In each case, we generate hints from the reasoning traces and inject them into the agent’s prompt to guide its decision-making. For MMLU, we evaluate the impact of these hints on answer accuracy, while for both AutoCodeRover and OpenHands, we

沃森（Watson）能够观察大语言模型的潜在推理轨迹，这些轨迹随后可由人类或另一个智能体进行分析或处理。

为支持这一演示，我们首先进行了一项初步的手动调试研究，在其中我们分析了由沃森生成的推理轨迹，以识别智能体行为中的失效模式。在我们的分析中，这些轨迹揭示了潜在的推理错误和错位假设，而这些在仅看最终答案时并不明显。基于这些发现，我们引入了一个自动化调试智能体，它消费相同的推理轨迹，以检测并干预有缺陷的推理路径。该智能体独立于主智能体和代理智能体运行，展示了沃森如何在修改底层模型或智能体的情况下实现语义可观测性。

虽然我们报告诸如 k improvement 通过率之类的指标来量化该自动化调试过程的结果，但这些结果应主要被解释为语义可观测性实用价值的证据。我们的目标不是评估沃森直接优化模型推理的能力，而是说明外部系统（无论是人工的还是自动化的）如何有效地利用由沃森生成的推理轨迹，以增强透明度并支持调试工作流。

B. 任务选择与研究设计

我们在非智能体任务和智能体任务上评估了沃森（Watson）的实用性，以展示其跨领域的泛化性和有效性。第一个任务涉及MMLU基准，这是一组多样化的选择题，涵盖专业法律、高中心理学和道德推理等领域。从全部 14,042道题中，我们随机抽取了三个子集，每个子集共 374道题，这对应于在95%置信水平和5%误差范围下达到统计显著性所需的最小题量。这些任务作为一个受控环境，用于观察大语言模型在智能体工作流之外的大语言模型行为。

作为第一步，我们提供了一个来自MMLU的定性示例，以说明沃森能够生成的推理类型。在列表1中，我们展示了一道来自专业法律类别的答错题目，其中智能体通过关注要约的持续时间选择了选项B，而非正确的选项A，并展示了沃森生成的相应推理轨迹。该推理轨迹（1）准确反映了主智能体所选的答案，（2）考虑了备选选项的优劣，（3）阐明了智能体决策的连贯的论证。这个例子突显了沃森即使在静态非智能体任务中也能生成详细且逻辑严谨的推理的能力。

列表1. exampleMMLU基准问题、初始错误答案以及由沃森Y生成的相关推理轨迹

You are a general knowledge quiz master. You are going to answer a question from a quiz. Only respond with the correct answer.
[问题] 商人销售货物的不可撤销的书面要约（实盘）(选项A) 如果受要约人提供包含该要约的格式合同，则必须另行签署。

B) 有效期为三个月。C) 不可转让。D) 即使支付对价，期限也不得超过三个月。[答案] B) 有效期为三个月。[推理] 该题询问的是商人销售货物的不可撤销书面要约的特征。选项A提到，如果受要约人提供包含该要约的格式合同，则该要约必须另行签署。该表述并未直接回答题目所问内容。选项B指出要约有效期为三个月。这似乎是一个可能的答案，因为它规定了要约可被视为有效的期限。选项C指出实盘不可转让。虽然这一点可能属实也可能不属实，但它与商人的不可撤销书面要约的特征无关。最后，选项D指出，即使支付对价，此类要约的期限也不得超过三个月。同样，虽然这一点可能属实也可能不属实，但它并未直接涉及构成商人销售货物的不可撤销书面要约的内容。基于这些选项并综合考虑回答该问题正确所需的所有因素，我认为选项B——有效期为三个月——是适当的响应，因为：(1) 它具体涉及此类要约的一项特征（效度期间）；(2) 它未提及题目要求之外的任何内容；(3) 其他选项的相关性均不够

为了在更复杂的真实场景下评估沃森，我们整合了 AutoCodeRover，这是一个自主软件工程智能体，它通过分析并修改源代码来解决GitHub风格的问题。我们将 AutoCodeRover应用于SWE-bench-lite，这是一个从更广泛的SWE-bench数据集 [28]中派生的包含300个问题的基准测试。AutoCodeRover分为两个主要阶段：首先，它从代码库中检索并综合相关上下文以识别可能的故障区域；其次，它生成代码补丁来修复问题。由于整体性能取决于智能体的初始代码定位步骤，我们的评估仅专注于这一不透明的第一阶段。

为了评估沃森在不同智能体脚手架上的泛化性，我们额外在相同的 SWE-bench-lite 基准测试上评估了 OpenHands。这使我们能够观察沃森是否也能在不同智能体架构和底层模型中，同样地捕捉潜在推理轨迹并引导调试。

为了在这一场景中定性地了解沃森的有效性，我们对来自 AutoCodeRover 评估的两个失败案例进行了手动调试分析。人类开发者审阅问题报告、原始 AutoCodeRover 输出以及沃森生成的推理轨迹（第三部分），以诊断智能体推理中可能的失误。基于这些见解，开发者随后制定了一个有针对性的决策优化提示，并将其与原始提示一起反馈给 AutoCodeRover，以重新尝试该问题。这项手动研究有助于评估沃森的追踪是否能实质性地帮助人类理解和纠正复杂的智能体行为。

基于这些定性发现，我们使用专门的调试智能体在 MMLU、AutoCodeRover 和 OpenHands 上开展了一项全面规模的自动化研究。在每种情况下，我们都从推理轨迹中生成提示，并将它们注入到智能体的提示中，以指导其决策。对于 MMLU，我们评估这些提示对答案准确率的影响；而对于 AutoCodeRover 和 OpenHands，则我们

Listing 2. The Watson-generated reasoning for instance `django__django-13401`, highlighting key reasoning steps. The agent incorrectly focuses on `model_inheritance/tests.py`, which contains subclasses B and C, and proposes changes there, rather than targeting the actual source of the bug. The common points across the provided explanations can be summarized as follows:

- **Identified Bug Location****: All reasonings consistently highlight that the issue is centered around the `model_inheritance/tests.py` file, which contains classes A, B, and C. This file is repeatedly mentioned as critical to understanding where potential bugs may lie.
- **Focus on Classes B and C****: There is a shared emphasis on examining classes B and C due to their inheritance from class A, suggesting that any issues likely originate from these subclasses or the abstract base class.
- **Method Investigations****: Several explanations point out specific methods (`__eq__`, `__lt__`, and `__hash__`) in relation to fields within these classes, indicating a need to analyze how equality comparisons are implemented for instances of these models.
- **Sufficient Context Established****: Multiple reasonings express confidence that enough context has been gathered regarding relevant files and classes without needing further information at this stage.
- **Search Strategies Proposed****: Some responses suggest utilizing search functions like `'search_method_in_class()'` or looking for similar cases using other search tools to uncover additional insights about how fields are compared across different models.

Listing 3. Segment of AutoCodeRover output for instance `django__django-13401` with a reasoning hint. The hint guides AutoCodeRover to the correct patch in `django/db/models/fields/__init__.py` and to rule out `tests.py` as the source.

[ASSISTANT] The issue reported is related to the equality comparison of two fields in different models. The author found that if they pull the fields of two models into a shared set, one will be de-duplicated away, because they compare as equal. The root of the issue is that they compare equal, as fields only consider `self.creation_counter` when comparing for equality. So, we need to adjust `__eq__` so that if the `field.model` is different, they will compare unequal. Similarly, it is probably wise to adjust `__hash__` and `__lt__` to match. When adjusting `__lt__`, it may be wise to order first by `self.creation_counter` so that cases not affected by this equality collision won't be re-ordered. In the experimental branch, there was one test that broke if ordered them by model first.

The fix for this issue is not located in any `tests.py` files, although they are a good place to start to get more context on the issue. We also found that the code snippets we searched for are mostly located in the file `django/db/models/fields/__init__.py`, which makes sense as this is where the `Field` class is defined. We can also see that the class A is inherited by classes B and C, and B and C are inherited by other classes in the codebase. This could be the reason why the issue is not located in any `'tests.py'` files, as it may be related to the inheritance of classes. Based on the context we have gathered, we can now start writing patches to solve the issue.

assess their effect on localization performance across the entire benchmark.

C. Manual Debugging with Reasoning Traces

To explore the practical utility of Watson in real-world debugging scenarios, we conducted a manual analysis using two failure cases from the AutoCodeRover evaluation set (i.e., cases where the incorrect code files were identified by the first agent as buggy locations). For each failure case, we began by presenting the corresponding issue report and the output of running AutoCodeRover on the task to a human developer, along with the associated reasoning trace generated by Watson (Section III). The developer then attempted to infer which aspects of the issue were emphasized or overlooked by the agent and whether these misalignments could plausibly explain the failure. Based on this analysis, we constructed a decision-refining hint and provided it, alongside the original input prompt, to AutoCodeRover in an attempt to re-solve the issue.

When executing these manual cases, we use the FIM configuration of Watson, and use GPT-3.5-turbo-instruct as the underlying model, as it is the only OpenAI model that explicitly supports the FIM capabilities required. In this setup, both the primary and surrogate agents share the same model and decoding parameters to ensure consistency. For reasoning validation, we employ the default prompt decomposer and set $n = 10$, meaning that the judge considers up to the 10 most- and 10 least-attributed prompt components to verify that the surrogate reasoning aligns with the aspects the agent attended to. We generate a minimum of 10 reasoning traces per task. To summarize the resulting reasoning populations and perform

related analysis tasks, we use OpenAI’s GPT-4o-mini model, selected for its improved reasoning quality and reliability.

1) *Instance `django__django-13401`*:¹ This issue, reported in the Django [10] web framework, concerns a bug where attribute fields from different models—each inheriting from the same parent model—are incorrectly treated as equal. The problem arises because field equality is determined solely by the `self.creation_counter` attribute. As a result, fields from models B and C, both of which inherit from model A, are considered equal despite belonging to different subclasses. This leads to unexpected behavior—for example, when fields from models B and C are added to a set, one may be erroneously deduplicated. The issue reporter proposes modifying the `__eq__`, `__hash__`, and `__lt__` methods to ensure that fields from different parent models are correctly treated as unequal.

Output of Reasoning: The reasoning generated from Watson for `django__django-13401`, shown in Listing 2, suggests the agent is focusing on the test files, specifically `model_inheritance/tests.py`, which is not the code location for where we expect the solution patch to be applied. Following this incorrect decision pathway, AutoCodeRover attempts to modify subclasses B and C, which are contained in `model_inheritance/tests.py`, as the fix for the encountered issue.

After Fixing with Hints: Based on the Watson-generated reasoning, we add the following hint at the end of the issue report and rerun AutoCodeRover.

¹<https://code.djangoproject.com/ticket/31750>

Listing 2. 沃森生成的推理，对应实例 `django__django-13401`，突出关键推理步骤。该智能体错误地关注 `model_inheritance/tests.py`，该文件包含子类 B 和 C，智能体在此处提出修改建议，而未针对 bug 的实际来源。所提供的解释中的共同点可以总结如下：1. ****识别出的 Bug 位置****：所有推理一致强调，问题集中在 `model_inheritance/tests.py` 文件，该文件包含 A 类、B 类和 C 类。该文件被反复提及，对于理解潜在 bug 可能所在的位置至关重要。2. ****重点关注 B 类和 C 类****：由于 B 类和 C 类继承自 A 类，因此有共同的推理上在于检查 B 类和 C 类，这表明问题很可能源自这些子类或抽象基类。3. ****方法调查****：多个解释指出，与这些类内的字段相关的特定方法（`__eq__`、`__lt__` 和 `__hash__`），表明需要分析如何为这些模型的实例实现相等性比较。4. ****建立了足够的上下文****：多个推理表示，关于相关文件和类的足够上下文信息已经收集完毕，在当前阶段不需要进一步的信息。5. ****搜索策略建议****：一些响应建议利用搜索函数（如 `'search_method_in_class()'`）或寻找类似案例的其他搜索工具来揭示有关字段比较的更多上下文，以揭示有关字段相等性的潜在问题。

清单 3. AutoCodeRover 针对实例 `django__django-13401` 的输出片段，带有推理提示。该提示引导 AutoCodeRover 定位到 `django/db/models/fields/__init__.py` 中的正确补丁，并排除 `tests.py` 作为来源。[助手] 所报告的问题与不同模型中两个字段的相等性比较有关。作者发现，如果将两个模型的字段放入一个共享集合中，其中一个字段会被去重掉，因为它们被比较为相等。问题的根源在于它们被比较为相等，因为字段在比较相等性时仅考虑 `self.creation_counter`。因此，我们需要调整 `__eq__`，以便当 `field.model` 不同时，它们将被比较为不相等。同样，调整 `__hash__` 和 `__lt__` 以保持一致也是明智的。在调整 `__lt__` 时，最好先按 `self.creation_counter` 排序，避免因重新排序而影响未受此相等性冲突影响的用例。在实验分支中，如果先按模型排序，其中一个测试会失败。该问题的修复方案并不位于任何 `tests.py` 文件中，尽管它们是获取有关问题更多上下文的好起点。我们还发现，我们搜索的代码片段大多位于文件 `django/db/models/fields/__init__.py` 中，这很有道理，因为这是 `Field` 类的定义位置。我们还能看到，A 类被 B 类和 C 类继承，而 B 类和 C 类又被代码库中的其他类继承。这可能是该问题未位于任何 `'tests.py'` 文件中的原因，因为它可能与类的继承有关。根据我们收集的上下文，我们现在可以开始编写补丁来解决这个问题。

评估它们对整个基准测试中定位性能的影响。

C. 利用推理轨迹进行手动调试

为了探索沃森在实际调试场景中的实用价值，我们使用 AutoCodeRover 评估集中的两个失败案例（即被第一个智能体错误地识别为有缺陷位置的文件对应的案例）进行了手动分析。对于每个失败案例，我们首先向人类开发者展示相应的问题报告以及 AutoCodeRover 在该任务上的运行输出，同时展示由沃森生成的相关推理轨迹（第 III 节）。随后，开发者试图推断该智能体强调或忽略了问题的哪些方面，以及这些不一致之处是否合理地解释了该失败。基于此分析，我们构建了一个决策优化提示，并将其与原始输入提示一起提供给 AutoCodeRover，以尝试重新解决该问题。

在执行这些手动案例时，我们使用沃森的 FIM 配置，并采用 GPT-3.5-turbo-instruct 作为底层模型，因为它是唯一明确支持所需 FIM 功能的 OpenAI 模型。在此配置中，主智能体和代理智能体共享相同的模型和解码参数，以确保一致性。对于推理验证，我们使用默认的提示分解器并设置 $n = 10$ ，这意味着评判器最多会考虑归因度最高的 10 个和归因度最低的 10 个提示组件，以验证代理推理是否与智能体关注的方面一致。我们为每个任务至少生成 10 条推理轨迹。为了概括所得的推理种群并执行

在相关的分析任务中，我们采用 OpenAI 的 GPT-4o-mini 模型，因其推理质量和可靠性得到提升而被选定。

1) 实例 `django __django-13401`:¹ 该问题报告于 Django [10] Web 框架，涉及一个缺陷：来自不同模型（各自继承自同一个父模型）的属性字段被错误地视为相等。问题的根源在于字段相等性仅由 `self.creation_counter` 属性决定。因此，来自模型 B 和模型 C 的字段（两者都继承自模型 A）尽管属于不同的子类，却被视为相等。这导致了意外行为——例如，当来自模型 B 和模型 C 的字段被添加到集合时，其中一个可能会被错误地去重。问题报告者建议修改 `__eq__`、`__hash__` 和 `__lt__` 方法，以确保来自不同父模型的字段被正确地视为不相等。

推理输出：由 Watson 为 `django__django-13401` 生成的推理，如列表 2 所示，表明智能体正聚焦于测试文件，特别是 `model_inheritance/tests.py`，这并不是我们期望应用解决方案补丁的代码位置。沿着这一错误的决策路径，AutoCodeRover 试图修改包含在 `model_inheritance/tests.py` 中的子类 B 和 C，以作为对遇到问题的修复。

使用提示修复后：基于沃森生成的推理，我们在问题报告末尾添加了以下提示，并重新运行 AutoCodeRover。

¹<https://code.djangoproject.com/ticket/31750>

Listing 4. Watson-generated reasoning for instance `django__django-14238`, showing focus on `example/core/models.py` while overlooking the issue’s reference to `__subclasscheck__`, absent from all explanations.

****Common Points Across Reasonings:****

- **Focus on "MyModel" and `__init__` Method**:** Multiple explanations consistently mention the class "MyModel" and its constructor method `__init__`, indicating that they are central to identifying the bug locations.
- **DEFAULT_AUTO_FIELD Issues**:** There is a recurring theme regarding problems with `DEFAULT_AUTO_FIELD`, suggesting it is linked to the bugs being analyzed.
- **File Location**:** The file "`example/core/models.py`" is frequently identified as a potential source of issues, reinforcing its importance in diagnosing the problem.
- **Need for Additional Context**:** Several reasonings highlight whether more context or information from other files/methods might be necessary, reflecting uncertainty about fully understanding all aspects of the issue.

The fix for this issue is not located in any “tests.py” files, although they are a good place to start to get more context on the issue.

With this hint, AutoCodeRover arrives at the correct file location to be patched, while specifically acknowledging that the fix is not located in any test files (Listing 3).

2) *Instance `django__django-14238`:*² This issue is reported in the Django framework and describes an application crash during model initialization when `DEFAULT_AUTO_FIELD` is set to a custom subclass of `BigAutoField`. The issue report includes a traceback demonstrating a `ValueError` requiring that the custom primary key class must subclass `AutoField`. The issue stems from a failure in `AutoFieldMeta.__subclasscheck__`, which does not recognize valid subclasses of `BigAutoField` or `SmallAutoField`. The issue report includes a suggested fix to modify the subclass check to include these subclasses in the `_subclasses` property.

Output of Reasoning: The reasoning generated from Watson for `django__django-14238`, shown in Listing 4, suggests that the agent is placing its focus on the files in the `example` directory. However, upon closer inspection of the original issue report, it is explicitly mentioned that the fix can be applied to the `__subclasscheck__` method, which does not show up at all in the reasonings, indicating that the agent is ignoring this information.

After Fixing with Hints: Based on the reasonings, we decide to add the following hint at the end of the issue report and rerun AutoCodeRover.

The fix for this issue would not be located in the ‘example’ directory. Make sure you focus on the ‘__subclasscheck__’ method.

With this hint, AutoCodeRover retrieves the correct contextual information related to `__subclasscheck__` and arrives at the correct file location to be patched (Listing 5).

D. Automated Debugging Pipeline

Building on the insights gained from the manual debugging study, we design a two-stage automated evaluation pipeline to demonstrate how Watson-generated reasoning traces can be effectively leveraged in downstream debugging and decision-making processes. Rather than aiming to directly improve

TABLE II
PASS@1 / PASS@3 ACCURACY ACROSS EVALUATION RUNS ON MMLU (374 QUESTIONS PER RUN). CONFIGURATIONS: RO-SR = RESPONSE-ONLY SELF-REFLECTION, WA-SR = WATSON-AIDED SELF-REFLECTION.

Configuration	Run 1	Run 2	Run 3
RO-SR (FIM)	59.4% / 68.2%	59.4% / 69.8%	60.7% / 71.1%
WA-SR (FIM)	58.8% / 66.8%	61.0% / 71.4%	63.3% / 73.5%
RO-SR (RepCoT)	63.6% / 70.9%	63.1% / 71.4%	66.3% / 74.3%
WA-SR (RepCoT)	61.2% / 73.0%	66.0% / 74.3%	67.9% / 77.2%

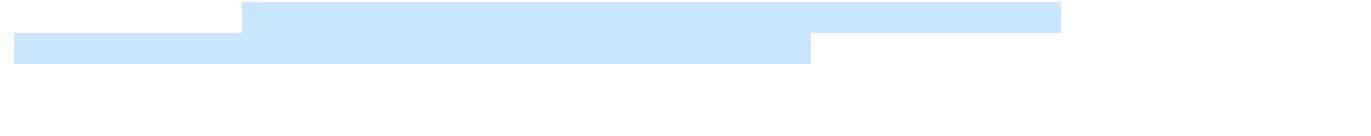
agent performance, this pipeline seeks to showcase the practical utility of semantic observability by comparing system behavior with and without Watson’s reasoning traces.

To clearly isolate the contribution of each component, we generate guiding hints for the agent under two distinct cases: (1) using Watson-generated reasoning traces along with the agent’s original answer, and (2) using only the agent’s original answer without any reasoning traces. This second case serves as an explicit ablation run, allowing us to measure the added value of incorporating the reasoning traces into the hint.

We begin by establishing a baseline through an ablation study. Each agentic system is first executed in its standard form (i.e., without the involvement of Watson) to produce an initial answer. For MMLU, this answer corresponds to the selected option for a benchmark question; for AutoCodeRover and OpenHands, it refers to the code location identified as faulty. We then simulate a self-reflection [40] process by prompting a separate LLM to evaluate whether the produced answer is reasonable in the context of the task (i.e., the multiple-choice question or the issue report). The reflection is performed without access to any reasoning traces, relying solely on the answer and task description. The agent is subsequently rerun on the task, with the self-reflection output injected into its prompt as a guiding hint. The correctness of the newly generated answer is then evaluated, and the result constitutes the baseline performance.

Next, we repeat the process with Watson applied to the same set of tasks. In this configuration, the surrogate agent generates a reasoning trace that explains how the primary agent arrived at its answer. This trace is incorporated into the self-reflection stage, enabling the reflection LLM to critique both the final answer and the reasoning path that led to it. As before, the resulting reflection is provided to the agent, which reruns the task with the added guidance. We then assess the

清单4. 沃森生成的针对实例`django__django-14238`的推理，其重点在于`example/core/models.py`，但忽略了问题中提及的`__subclasscheck__`，且该提及未出现在任何解释中。****各推理中的共同点：****** 1. **关注 "MyModel" 和`__init__`方法**:** 多个解释一致提到了类 "MyModel" 及其构造函数`__init__`，表明它们是确定bug位置的核心。**2. **DEFAULT_AUTO_FIELD问题**:** 关于`DEFAULT_AUTO_FIELD`存在反复出现的主题，暗示其与正在分析的bug相关。**3. **文件位置**:** 文件 "`example/core/models.py`" 经常被识别为潜在的问题来源，强化了其在诊断问题中的重要性。**4. **需要更多上下文**:** 多个推理强调了可能需要来自其他文件/方法的更多上下文或信息，反映了对完全理解问题所有方面的不确定性。



该问题的修复并不位于任何“*tests.py*”文件中，尽管它们是获取有关该问题更多上下文的良好起点。

借助此提示，AutoCodeRover 能够到达需要修补的正确文件位置，同时特别指出修复方案并不位于任何测试文件中（Listing3）。

2) 实例 `django__django-14238`:² 此问题报告是在 Django 框架中提交的，描述了在将 `DEFAULT_AUTO_FIELD` 设置为 `BigAutoField` 的自定义子类时，模型初始化期间发生的应用程序崩溃。问题报告包含一个回溯（`traceback`），显示 `ValueError`，要求自定义主键类必须继承自 `AutoField`。该问题源于 `AutoFieldMeta.__subclasscheck__` 中的失败，它无法识别 `BigAutoField` 或 `SmallAutoField` 的有效子类。问题报告还包含一个建议的修复方案，即修改子类检查，将这些子类包含在 `_subclasses` 属性中。

推理输出：沃森针对 `django__django-14238` 生成的推理（如 Listing 4 所示）表明，该智能体正将注意力集中在 `example` 目录中的文件上。然而，更仔细地检查原始问题报告后会发现，其中明确提到修复方案可以应用于 `__subclasscheck__` 方法，而该方法在推理中完全没有出现，这表明该智能体忽略了此信息。

在利用提示修复后：基于上述推理，我们决定在问题报告末尾添加以下提示，并重新运行 AutoCodeRover。

此问题的修复不会位于“*example*”目录中。请确保重点关注“*subclasscheck*”方法。

借助此提示，AutoCodeRover 检索到与 `subclasscheck` 和 `__init__` 相关的正确上下文信息，并确定了需要修补的正确文件位置（列表 5）。

D. 自动调试流水线

基于手动调试研究中获得见解，我们设计了一个两阶段的自动化评估流水线，以展示沃森生成的推理轨迹如何在下游调试和决策过程中得到有效利用。我们的目标并非直接提升

表 II MMLU（每轮 374 题）评估运行中 `pass@1 / pass@3` 的准确率。配置：RO-SR = 仅响应自反思，WA-SR = Watson辅助自反思。

配置	运行 1	运行 2	运行 3
RO-SR (FIM)	59.4% / 68.2%	59.4% / 69.8%	60.7% / 71.1%
WA-SR (FIM)	58.8% / 66.8%	61.0% / 71.4%	63.3% / 73.5%
RO-SR (RepCoT)	63.6% / 70.9%	63.1% / 71.4%	66.3% / 74.3%
WA-SR (RepCoT)	61.2% / 73.0%	66.0% / 74.3%	67.9% / 77.2%

在智能体性能方面，该流水线旨在通过比较使用与不使用沃森的推理轨迹时的系统行为，展示语义可观测性的实用价值。

为清晰隔离各组件的贡献，我们在两种不同情况下为智能体生成指导提示：（1）使用沃森生成的推理轨迹以及智能体的原始答案；（2）仅使用智能体的原始答案，不使用任何推理轨迹。第二种情况作为明确的消融实验，使我们能够衡量将推理轨迹纳入提示所带来的附加价值。

我们首先通过消融研究建立基线。每个智能体系统首先以标准形式（即不涉及沃森）运行，以生成初始答案。对于 MMLU，该答案对应于基准问题的所选选项；对于 AutoCodeRover 和 OpenHands，它指的是被识别为存在故障的代码位置。随后，我们通过提示一个独立的大语言模型来模拟自我反思 [40] 过程，以评估所生成的答案在任务上下文（即多选题或问题报告）中是否合理。反思过程在不访问任何推理轨迹的情况下进行，仅依赖答案和任务描述。随后，智能体在该任务上重新运行，自我反思的输出被注入到其提示中作为指导提示。然后评估新生成答案的正确性，该结果构成基线性能。

接下来，我们将沃森应用于同一组任务，并重复该过程。在此配置下，代理智能体生成一条推理轨迹，解释主智能体是如何得出其答案的。该轨迹被纳入自我反思阶段，使反思大型语言模型能够同时评估最终答案以及导致该答案的推理路径。与之前一样，得到的反思结果会被提供给智能体，智能体随后在额外指导的辅助下重新执行任务。然后我们评估

²<https://code.djangoproject.com/ticket/32620>

³<https://code.djangoproject.com/ticket/32620>

Listing 5. Segment of AutoCodeRover output for instance django__django-14238 with a reasoning hint, enabling AutoCodeRover retrieve context on __subclasscheck__ and locate the correct patch in django/db/models/fields/__init__.py, avoiding prior focus on the example directory. [ASSISTANT] Based on the search results, we can see that there are 12 classes with the name MyModel in the codebase, but none of them are located in the "example" directory. This means that the issue is not related to the "example" project. We also found 1 method with the name __subclasscheck__, which is located in the django/db/models/fields/__init__.py file. This method is inherited from the AutoFieldMeta class and is used to check if a subclass is a subclass of AutoField. Additionally, we found 3 snippets containing 'DEFAULT_AUTO_FIELD =' in the codebase. These snippets are located in the tests /test_sqlite.py, tests/check_framework/test_model_checks.py, and django/conf/global_settings.py files. These snippets are used to set the default primary key field type in the codebase. Finally, we found 2 snippets containing '__subclasscheck__' in the codebase, both of which are located in the django/db/models/fields/__init__.py file. These snippets are used to check if a given subclass is a subclass of AutoField. Based on this context, we can conclude that the issue is related to the django/db/models/fields/ __init__.py file, specifically the AutoFieldMeta class and its __subclasscheck__ method. The fix for this issue would not be located in the "example" project, but rather in the django project itself.

correctness of this revised output. In summary, for each task, we generate two hints: one that leverages Watson’s reasoning trace (the “hint from reasoning+answer” case) and one that does not (the ablation “hint from answer only” case).

Building on this setup, we systematically evaluate each task under both ablation (without Watson) and Watson-enabled conditions. For the MMLU evaluation, the Watson-enabled pipeline is tested under both configurations of the Watson framework: FIM, using GPT-3.5-turbo-instruct as the underlying language model, and RepCoT, using GPT-3.5-turbo-0125, which offers chat-based interaction while maintaining close behavioral similarity to GPT-3.5-turbo-instruct. For the AutoCodeRover experiment, we use the FIM configuration with GPT-3.5-turbo-instruct, while for the OpenHands experiment, we use the RepCoT configuration with Qwen3-8B. In both configurations, the primary and surrogate agents still share the same underlying model and are executed using identical decoding parameters. For reasoning validation, we also employ the default prompt decomposer and set $n = 10$, such that the judge examines the 10 highest- and lowest-attributed prompt components to verify alignment between the surrogate reasoning and the agent’s attention. We generate a minimum population size of 10 reasonings. Finally, tasks within the evaluation pipeline (e.g., summarization of reasoning populations, hint generation, and grading outputs) are performed using OpenAI’s GPT-4o-mini model, selected for its improved reasoning quality and reliability in downstream analysis tasks. We compute pass@1 and pass@3 for MMLU, and pass@1 for AutoCodeRover and OpenHands.

E. Evaluation Results

This section presents the empirical results of applying the Watson framework across MMLU, AutoCodeRover, and OpenHands. Our evaluation compares agent performance under two conditions: (1) a self-reflection pipeline without access to Watson reasoning traces, and (2) a self-reflection pipeline enhanced with Watson-generated reasoning traces.

1) *MMLU Results:* Across three statistically significant evaluation runs on the MMLU benchmark (Section IV-B), we investigate whether incorporating Watson-generated reasoning traces supports the surrogate agent’s self-reflection process. Rather than demonstrating uniform improvements in accuracy, our results, shown in Table II, highlight nuanced shifts in

performance metrics that suggest the potential value of incorporating Watson-generated reasoning into the reflection evaluation process. These effects are observed across both the FIM and RepCoT configurations of Watson.

In the FIM setting, the baseline Pass@1 accuracy across the three runs ranges from 59.4% to 60.7%, while incorporating reasoning traces increases this to a range of 58.8% to 63.4%. Although Pass@1 performance shows marginal variance in the first experiment, the gains become more substantial in subsequent runs. The improvements are more consistently observed in Pass@3 accuracy, which increases from 68.2%-71.1% in the baseline to 66.8%–73.5% with reasoning, with each of the three runs demonstrating improvement. The χ^2 test shows no significant differences (Pass@1 $p=0.88,0.65,0.45$; Pass@3 $p=0.50,0.63,0.46$), and overlapping 95% CIs indicate gains are trends, not definitive improvements.

For the RepCoT configuration, the baseline Pass@1 scores range from 63.1% to 66.3%, while the inclusion of reasoning increases them to 61.2%–67.9%, with the largest improvement observed in the third run. More prominently, Pass@3 accuracy improves from 70.9–74.3% (baseline) to a range of 73.0–77.2% when Watson-generated reasoning is used. These results indicate that Watson’s explicit reasoning generation capabilities enhance the agent’s self-reflective judgment, especially in settings where structured reasoning like RepCoT is applied. As with the FIM setting, the χ^2 test shows no significant differences for RepCoT (Pass@1 $p=0.50,0.40,0.64$; Pass@3 $p=0.52,0.37,0.35$), and overlapping 95% CIs indicate gains are trends, not definitive improvements. However, the gains in Pass@3 suggest that reasoning traces enable the agent to better recognize partial correctness and near-miss solutions, improving its overall evaluative reliability.

2) *AutoCodeRover and OpenHands Results:* For both AutoCodeRover and OpenHands, we focus our evaluation on tasks where the agent successfully identified a buggy location. Each case is categorized as *correct* if the located code region matches the ground truth and *incorrect* otherwise; cases where the agent fails to identify any location are excluded from analysis. This approach allows us to assess the impact of Watson-generated reasoning on downstream debugging accuracy without being confounded by runs that terminate prematurely. Given the computational cost and practical constraints, we report pass@1 for both agents to provide a tractable yet meaningful measure evaluation.

清单 5. AutoCodeRover 针对实例 django__django-14238 的输出片段，带有推理提示，使 AutoCodeRover 能够获取 __subclasscheck__ 的上下文，并在 django/db/models/fields/__init__.py 中定位正确的补丁，从而避免先前对 example 目录的关注。[ASSISTANT] 根据搜索结果，我们可以看到代码库中有 12 个名为 MyModel 的类，但都不在 "example" 目录中。这意味着该问题与 "example" 项目无关。我们还发现了一个名为 __subclasscheck__ 的方法，它位于 django/db/models/fields/__init__.py 文件中的 AutoFieldMeta 类，用于检查一个子类是否为 AutoField 的子类。此外，我们在代码库中找到了 3 个包含 'DEFAULT_AUTO_FIELD =' 的片段。这些片段位于 tests /test_sqlite.py、tests/check_framework/test_model_checks.py 和 django/conf/global_settings.py 文件中。这些片段用于设置代码库中默认的主键字段类型。最后，我们在代码库中找到了 2 个包含 '__subclasscheck__' 的片段，它们都位于 django/db/models/fields/__init__.py 文件中。这些片段用于检查给定的子类是否为 AutoField 的子类。根据此上下文，我们可以得出结论，该问题与 django/db/models/fields/__init__.py 文件有关，特别是 AutoFieldMeta 类及其 __subclasscheck__ 方法。该问题的修复不应位于 "example" 项目中，而应位于 django 项目本身。

此修订输出的正确性。总之，对于每个任务，我们生成两个提示：一个利用沃森的推理轨迹（即“来自推理+答案的提示”情形），另一个则不利用（即消融实验中的“仅来自答案的提示”情形）。

基于该设置，我们在消融实验（不含沃森）和沃森增强条件下，系统地评估了每一项任务。对于 MMLU 评估，沃森增强流水线在沃森框架的两种配置下均进行了测试：FIM 配置，使用 GPT-3.5-turbo-instruct 作为底层语言模型；以及 RepCoT 配置，使用 GPT-3.5-turbo-0125，该模型提供基于聊天的交互方式，同时保持与 GPT-3.5-turbo-instruct 高度相似的行为特性。对于 AutoCodeRover 实验，我们采用使用 GPT-3.5-turbo-instruct 的 FIM 配置；而对于 OpenHands 实验，我们采用使用 Qwen3-8B 的 RepCoT 配置。在两种配置中，主智能体与代理智能体仍共享相同的底层模型，并使用相同的解码参数执行。对于推理验证，我们还使用默认的提示分解器并设置 $n = 10$ ，以便评判器检查归因最高的 10 个和最低的 10 个提示组件，从而验证代理推理与智能体的注意力之间的对齐性。我们生成至少 10 个推理样本作为最小总体规模。最后，评估流水线内的任务（例如推理总体的摘要、提示生成和输出评分）使用 OpenAI 的 GPT-4o-mini 模型执行，选择该模型是因为其在下游分析任务中具有更高的推理质量和可靠性。我们计算 MMLU 的 pass@1 和 pass@3，以及 AutoCodeRover 和 OpenHands 的 pass@1。

E. 评估结果

本节展示了将沃森框架应用于 MMLU、AutoCodeRover 和 OpenHands 的实验结果。我们的评估比较了智能体在两种条件下的性能：（1）无沃森推理轨迹访问权的自我反思管道，以及（2）使用沃森生成的推理轨迹增强的自我反思管道。

1) *MMLU 基准测试的结果：*在三次具有统计显著性的 MMLU 基准测试评估运行中（第四节 B），我们研究了纳入沃森生成的推理追踪是否有助于代理智能体的自我反思过程。我们的结果（如表 II 所示）并未显示出准确率的普遍提升，而是突出了细微的变化，

这些指标表明将沃森生成的推理纳入反思评估过程可能具有潜在价值。这些效果在沃森的 FIM 和 RepCoT 两种配置中均有观察。

在 FIM 设置中，基线的 pass@1 准确率在三次运行中处于 59.4% 至 60.7% 的区间，而引入推理轨迹后，该区间变为 58.8% 至 63.4%。尽管在第一次实验中，pass@1 性能仅表现出轻微差异，但在后续运行中，提升变得更为显著。在 pass@3 准确率上，改进表现更为一致，其数值从基线的 68.2%–71.1% 提升至加入推理后的 66.8%–73.5%，且这三次运行均显示出提升。 χ^2 检验未显示出显著差异（Pass@1 $p=0: 0.88, 0.65, 0.45$; Pass@3 $p=0: 0.50, 0.63, 0.46$ ），重叠的 95% 置信区间表明这些增益是趋势，而非确定性改进。

对于 RepCoT 配置，基线的 pass@1 得分范围为 63.1% 至 66.3%，而引入推理后，得分提升至 61.2%–67.9%，其中最大的提升出现在第三次运行中。更为显著的是，当使用沃森生成的推理时，pass@3 的准确率从 70.9–74.3%（基线）提升至 73.0–77.2%。这些结果表明，沃森的显式推理生成能力增强了智能体的自我反思判断，尤其是在应用了如 RepCoT 这类结构化推理的环境中。与 FIM 配置类似，针对 RepCoT 的 χ^2 测试显示没有显著差异（pass@1 $p=0.50,0.40,0.64$; pass@3 $p=0.52,0.37,0.35$ ），且重叠的 95% 置信区间表明这些增益是趋势，而非确定性的提升。然而，pass@3 的提升表明，推理轨迹使智能体能够更好地识别部分正确和近乎正确的解决方案，从而提高其整体评估的可靠性。

2) *AutoCodeRover 和 OpenHands 的结果：*对于 AutoCodeRover 和 OpenHands，我们将评估重点放在智能体成功定位缺陷位置的任务上。如果定位的代码区域与真实值匹配，则该案例被归类为正确，否则为错误；智能体未能定位到任何位置的案例将被排除在分析之外。这种方法使我们能够评估 Watson 生成的推理对下游调试准确率的影响，同时避免被过早终止的运行所混淆。鉴于计算成本和实际约束，我们报告了两种智能体的 pass@1，以提供一个可行且有意义的评估指标。

Across the two agentic experiments on SWE-bench-lite, we focus on tasks where the agent successfully identified potential buggy code locations. For AutoCodeRover, 103 valid tasks were analyzed: the baseline self-reflection pipeline achieved a Pass@1 of 71, while the Watson-enabled pipeline reached 75. For OpenHands, 57 valid tasks were considered, with a baseline Pass@1 of 33 and a Watson-enabled Pass@1 of 34 (Table III). In both cases, the slight improvements observed with Watson-generated reasoning were not statistically significant ($p > 0.05$), indicating trends rather than definitive gains in localization accuracy.

TABLE III
PASS@1 ACCURACY OF SELF-REFLECTION PIPELINES ON VALID AUTOCODEROVER AND OPENHANDS SAMPLES (SWE-BENCH-LITE).
RO-SR = RESPONSE-ONLY SELF-REFLECTION, WA-SR = WATSON-AIDED SELF-REFLECTION.

Configuration	Valid Samples	RO-SR	WA-SR
AutoCodeRover	103	71 (68.9%)	75 (72.8%)
OpenHands	57	33 (57.9%)	34 (59.6%)

F. Comparison with Native Reasoning Models

To further evaluate the quality of Watson’s post-hoc reasoning traces, we compared them against traces produced natively by a reasoning-capable large language model. We selected Qwen3-8B, which provides a “thinking” mode to explicitly generate reasoning alongside answers. Using a statistically representative MMLU sample from Section IV-B, we ran the model twice per example: once with “thinking” enabled to collect native reasoning, and once with reasoning disabled to obtain only the final answer. In the latter case, we applied Watson to the input–output pair to generate post-hoc reasoning traces using the RepCoT configuration, as Qwen3-8B does not support FIM.

Each Watson-generated trace was paired with its corresponding native Qwen3-8B trace. We used an LLM-as-a-judge to assess whether the two traces were semantically equivalent, focusing on whether they followed the same logical progression, invoked similar assumptions, and reached conclusions through comparable reasoning, while ignoring superficial wording differences.

For Qwen3-8B, we compared Watson’s post-hoc reasoning traces with the model’s natively generated reasoning traces, conditioned on whether both runs produced the same final answer. Across the sampled MMLU tasks, the two runs produced the same answer in 307 cases (76.2%) and diverged in 67 cases (16.6%). When the answers matched, the reasoning traces were semantically equivalent in 212 cases (69.1%) and diverged in 95 cases (30.9%). Conversely, when the answers differed, the reasoning traces almost always diverged (64 cases, 95.5%), with only a small number of coincidental overlaps (3 cases, 4.5%, $p < 10^{-20}$, Fisher’s exact test). While the divergence observed in roughly one-third of the matching-answer cases might initially appear as a limitation, it could in fact be advantageous: recent work has shown that natively generated

reasoning traces are not always faithful to the model’s actual inference process, often reflecting superficial rationalizations rather than genuine decision pathways [3, 8]. By contrast, Watson enforces consistency with the model’s saliency during inference, allowing it to surface potential discrepancies between an agent’s stated reasoning and its underlying behaviour. In this sense, divergences revealed by Watson may highlight cases where post-hoc reasoning provides a truer window into the model’s decision-making, offering a valuable diagnostic signal for debugging and analysis.

G. Summary

This case study demonstrates the utility of Watson as a semantic observability framework for supporting both manual and automated debugging of LLM-powered agents. Across static (MMLU) and dynamic (AutoCodeRover and OpenHands) tasks, we find that Watson-generated reasoning traces consistently surface actionable signals that aid fault localization and enable targeted interventions. Our findings reinforce that Watson enables insight into agent reasoning without modifying the primary agent or underlying model. However, the debugging agents used in this study are illustrative rather than optimized, designed to validate trace utility rather than maximize performance. Moreover, Watson is an observability layer, not a reasoning optimizer, and its effectiveness ultimately depends on the interpretability and completeness of the extracted traces.

V. APPLICATIONS, LIMITATIONS, AND THREATS TO VALIDITY

In this section, we discuss potential applications of Watson in practical development settings and examine its limitations and threats to validity.

A. Applications in Practical Settings

A promising avenue for the adoption of Watson is its integration into observability platforms designed for agentic workflows. In such settings, workflow executions are typically instrumented to capture detailed traces of agent behavior, including inputs, outputs, and intermediate decision steps. Watson can be incorporated into these platforms as an on-demand reasoning reconstruction service, enabling users to request surrogate reasoning for specific agent calls when deeper insight is required. This capability extends traditional observability by augmenting traces with interpretable reasoning paths, thereby facilitating debugging, auditing, and comprehension of complex multi-agent workflows. Crucially, Watson achieves this without modifying or delaying the execution of the original agent, making it suitable for deployment in developer environments (e.g., IDEs), continuous integration and deployment pipelines, or production-grade observability dashboards. Such integration illustrates Watson’s potential to complement existing software engineering practices with cognitive observability, thereby improving transparency and trust in LLM-powered agents.

In针对 SWE-bench-lite 的两项智能体实验中，我们聚焦于智能体成功识别出潜在缺陷代码位置的任务。对于 AutoCodeRover，共分析了 103 个有效任务：基线自我反思管道的 Pass@1 为71，而沃森增强管道的 Pass@1 达到75。对于 OpenHands，共考虑了57个有效任务，其基线 Pass@1 为33，沃森增强后的 Pass@1 为34（表 III）。在这两种情况下，使用沃森生成的推理所观察到的微小提升均不具有统计学显著性（ $p > 0.05$ ），这表明这仅是一种趋势，而非定位准确率方面的明确提升。

表 III 在 AutoCodeRover 和 OpenHands 的有效样本上自我反思管道的 pass@1 准确率（SWE-bench-lite）。RO-SR = 仅响应自反思，WA-SR = 沃森辅助自反思。

配置	有效样本	RO-SR	WA-SR
AutoCodeRover	103	71 (68.9%)	75 (72.8%)
OpenHands	57	33 (57.9%)	34 (59.6%)

F. 与原生推理模型的对比

为了进一步评估沃森的事后推理轨迹的质量，我们将这些轨迹与由具备推理能力的大语言模型原生生成的轨迹进行了比较。我们选择了 Qwen3-8B，该模型提供“思考”模式，用于在生成答案的同时显式生成推理过程。使用第四部分 B 节中具有统计代表性的 MMLU 样本，我们对每个 example 运行了该模型两次：一次启用“思考”模式以收集原生推理，另一次禁用推理以获得最终答案。在后一种情况下，我们对输入–输出对应用了沃森，并使用 RepCoT 配置生成事后推理轨迹，因为 Qwen3-8B 不支持 FIM。

每条由沃森生成的推理轨迹都与其对应的 Qwen3-8B 原生轨迹配对。我们使用 LLM 作为评判器来评估这两条推理轨迹是否语义等价，重点关注它们是否遵循相同的逻辑顺序，是否引用了相似的假设，并通过可比的推理得出结论，同时忽略表面措辞上的差异。

对于 Qwen3-8B，我们将沃森的事后推理轨迹与模型原生生成的推理轨迹进行了比较，并以两次运行是否产生相同的最终答案作为条件。在采样的 MMLU 任务中，两次运行在 307 个案例中（76.2%）产生了相同的答案，而在 67 个案例中（16.6%）存在分歧。当答案一致时，推理轨迹在 212 个案例中（69.1%）语义等价，而在 95 个案例中（30.9%）存在分歧。相反，当答案不同时，推理轨迹几乎总是存在分歧（64 个案例，95.5%），仅有极少数情况出现偶然的重合（3 个案例，4.5%， $p < 10^{-20}$ ，费雪精确检验）。尽管在大约三分之一的答案一致案例中观察到的分歧最初可能被视为一种局限性，但实际上它可能具有优势：近期研究表明，原生生成的

推理轨迹并不总是忠实于模型的实际推理过程，往往反映的是表面的合理解释，而非真正的决策路径 [3, 8]。相比之下，沃森在推理过程中强制保持与模型显著性的一致性，使其能够揭示智能体所陈述的推理与其潜在行为之间的潜在差异。从这个意义上说，沃森揭示的差异可能突显出事后推理能更真实地反映模型决策的情况，为调试和分析提供有价值的诊断信号。

G. 摘要

本案例研究展示了沃森作为语义可观测性框架在支持由大语言模型驱动的智能体的手动和自动调试方面的效用。在静态（MMLU）和动态（AutoCodeRover 和 OpenHands）任务中，我们发现沃森生成的推理轨迹一致地揭示了有助于故障定位并实现针对性干预的可操作信号。我们的研究结果进一步证实，沃森能够在不修改主智能体或底层模型的情况下，提供对智能体推理的洞察。然而，本研究使用的调试智能体具有示意性而非经过优化，旨在验证推理轨迹的效用，而非最大化性能。此外，沃森是一个可观测性层，而非推理优化器，其有效性最终取决于所提取推理轨迹的可解释性和完整性。

V. 应用、局限性与效度威胁

在本节中，我们讨论了沃森在实际开发环境中的潜在应用，并考察了其局限性及对效度的威胁。

A. 实际应用环境中的应用

沃森（Watson）落地应用的一条有前景的路径是将其集成到为智能体 workflow 设计的可观测性平台中。在此类场景中，workflow 执行通常会经过插桩处理，以捕获智能体行为的详细追踪，包括输入、输出和中间决策步骤。沃森可以作为按需的推理重建服务集成到这些平台中，使得当需要更深入的洞察时，用户可以针对特定的智能体调用请求代理推理。这一能力通过用可解释的推理路径增强追踪，从而扩展了传统可观测性，促进了复杂多智能体工作流的调试、审计和理解。关键在于，沃森在不修改或不延迟原始智能体执行的情况下实现这一点，因此适合部署在开发环境（例如集成开发环境）、持续集成与部署管道或生产级可观测性仪表盘。这种集成展示了沃森在现有软件工程中补充认知可观测性的潜力，从而提高了由大语言模型驱动的智能体的透明度和可信度。

B. Limitations

While the integration scenarios highlight **Watson**’s potential value, it is important to consider its performance trade-offs. **Watson** adds no runtime overhead to the primary agent, since reasoning reconstruction is applied post hoc to captured traces. However, generating multiple completions with the surrogate agent to recover diverse reasoning paths and produce a meta-reasoning summary (Section III-A) incurs computational cost. In our experiments, we generated 10 reasonings per instance to balance coverage and expense. **Watson**’s computational cost then scales with the number of reasoning traces generated and the computation of prompt attribution scores used for trace verification, which scale with prompt length and component decomposition. Both FIM and RepCoT discard traces misaligned with these scores, regenerating until the desired number of valid traces is reached, further contributing to overhead.

Importantly, the number of surrogate completions provides a direct trade-off mechanism. Fewer completions reduce cost and latency but may miss important reasoning paths, while more completions increase coverage and confidence at the expense of computational resources. However, these computations do not affect the runtime of the observed agent, as they can be performed in parallel or post hoc. The exact cost depends on prompt complexity, the number of surrogate completions, and prompt segmentation for attribution scoring, allowing **Watson** to adapt to different deployment contexts, from lightweight development workflows to resource-intensive audits. Despite this flexibility, **Watson** cannot guarantee complete coverage of all plausible reasoning paths. Even with higher sampling, some trajectories may remain unexplored, especially for complex tasks with large search spaces. Consequently, the reconstructed reasoning should be interpreted as a representative sample rather than an exhaustive account of the agent’s internal processes.

A further limitation of **Watson** arises from its reliance on an LLM-as-a-judge for reasoning validation (Section III-D). After PromptExp identifies the most and least influential prompt components, the judge checks that each generated reasoning trace aligns with high-attribution components without overemphasizing low-attribution ones. While this grounds validation in the agent’s decision factors, the judge can misclassify traces, either rejecting plausible reasoning or accepting partially misaligned ones. We mitigate this by using a strong judge model and combining attribution scores with explicit rules (Section III-D), but some subjectivity and bias remain. The fidelity of verified traces thus depends on both surrogate coverage and judge evaluation, highlighting directions for improved calibration, robustness, and reliability in future work.

Finally, it is important to note that **Watson** has so far only been evaluated on text-based reasoning agents, and its applicability to other types of agents remains uncertain. Extending **Watson** to these domains would likely require additional mechanisms for capturing and interpreting non-textual context, as well as adapting the verification procedures

to handle richer forms of agent reasoning. Consequently, the framework’s generalizability beyond text-based reasoning agents should be considered a limitation and an area for future research.

C. Threats to Validity

Several factors may influence the reliability and generalizability of the experimental results reported for **Watson**. First, the fidelity of the surrogate agent is critical: if the surrogate does not accurately mirror the primary agent’s reasoning behavior, the reconstructed traces may not reflect the true internal processes, limiting the validity of our conclusions. Second, the evaluation depends on the LLM-as-a-judge used for reasoning verification. Biases or misclassifications by this model can affect which reasoning traces are retained, potentially influencing the resulting meta-reasoning summaries. Third, the framework has only been evaluated on text-based reasoning agents, and it remains uncertain how well the results would generalize to multimodal, embodied, or highly interactive agents. Finally, reproducibility is a potential concern: variations in surrogate configurations, the number of completions generated, or the choice of judge model may yield different reasoning reconstructions, affecting consistency across experiments. By explicitly acknowledging these threats, we aim to provide a transparent assessment of the contexts in which **Watson**’s results can be confidently interpreted.

VI. CONCLUSION

In this paper, we introduced the concept of *cognitive observability* as a crucial advancement in the observability of agentic software powered by foundation models. Traditional observability techniques fall short in these systems due to the opaque, non-deterministic reasoning processes of LLMs. To address this challenge, we proposed and implemented **Watson**, a novel framework that retrospectively recovers implicit reasoning traces with high fidelity while preserving the efficiency of standard LLMs.

Our evaluation demonstrates that **Watson** enables both manual debugging and automated runtime correction, effectively exposing the “why” behind agent decisions in a manner that was previously inaccessible without affecting the agent’s behavior or outputs. Through empirical validation on real-world agent tasks, including MMLU, and AutoCodeRover and OpenHands on the SWE-bench-lite benchmarks, **Watson** facilitates improved reasoning transparency and supports enhanced reliability and controllability of Agentware. Overall, **Watson** offers a practical step toward addressing the observability challenges of LLM-powered agents, helping developers better interpret and improve agent behavior. Future work should explore broader generalization across agent types and tasks, more sophisticated use of reasoning traces for intervention or retraining, and real-time observability in production environments. As LLMs continue to take on complex roles in Agentware, advancing tools for cognitive observability will be critical to supporting their safe and effective deployment.

B. 局限性

虽然集成场景凸显了沃森的潜在价值，但重要的是要考虑其性能权衡。沃森不会对主智能体增加任何运行时开销，因为推理重建是事后应用于捕获的追踪的。然而，使用代理智能体生成多个补全以恢复多样的推理路径并生成元推理摘要（第III-A节）会产生计算成本。在我们的实验中，我们为每个实例生成了10个推理，以平衡覆盖率和成本。沃森的计算成本随后会随着生成的推理追踪数量以及用于轨迹验证的提示归因分数的计算而增加，后者会随着提示长度和组件分解而扩展。FIM和RepCoT都会丢弃与这些分数不一致的追踪，并重新生成直到达到所需的数量有效追踪，这进一步增加了开销。

重要的是，代理补全的数量提供了一种直接的权衡机制。较少的补全会降低成本和延迟，但可能会遗漏重要的推理路径；而更多的补全则以计算资源为代价，提高覆盖率和置信度。然而，这些计算不会影响所观测智能体的运行时，因为它们可以并行或事后执行。具体成本取决于提示的复杂性、代理补全的数量以及用于归因评分的提示分割，这使得沃森能够适应不同的部署上下文，从轻量级的开发工作流到资源密集型审计。尽管具备这种灵活性，沃森仍无法保证对所有合理推理路径的完全覆盖。即使采用更高的采样率，某些轨迹可能仍未被探索，特别是在具有较大搜索空间的复杂任务中。因此，重构的推理应被视为智能体内部过程的一个代表性样本，而非详尽的账户。

沃森的另一项局限性源于其对以LLM作为裁判进行推理验证的依赖（第III-D节）。在PromptExp识别出影响最大和最小的提示组件之后，评判器会检查每个生成的推理轨迹是否与高归因组件保持一致，同时避免过度强调低归因项。虽然这种验证机制基于智能体的决策因素，但评判器可能会错误分类追踪，例如拒绝合理的推理或接受部分不一致的追踪。我们通过使用强大的评判模型，并将归因分数与显式规则相结合（第III-D节）来缓解这一问题，但仍存在一定的主观性和偏见。因此，已验证的轨迹的保真度既依赖于代理覆盖范围，也依赖于评判评估，这凸显了未来工作中需要改进校准、鲁棒性和可靠性等方向。

最后，值得注意的是，沃森（Watson）目前仅在基于文本的推理智能体上进行了评估，其对其他类型智能体的适用性尚不确定。将沃森扩展至这些领域，可能需要额外的机制来捕获和解释非文本上下文，并对验证流程进行适配

以处理更丰富的智能体推理形式。因此，应将该框架在基于文本的推理智能体之外的泛化性视为一项局限，并作为未来研究的领域。

C. 效度威胁

多个因素可能影响报告中沃森实验结果的可靠性和可推广性。首先，代理智能体的保真度至关重要：如果代理智能体未能准确反映主智能体的推理行为，则重构的推理轨迹可能无法反映真实的内部过程，从而限制我们结论的效度。其次，评估依赖于用于推理验证的“大语言模型作为评判器”这一机制。该模型存在的偏差或误分类可能影响哪些推理轨迹被保留，进而潜在地影响最终的元推理摘要。第三，该框架仅在基于文本的推理智能体上进行了评估，其结果能否有效推广到多模态、具身或高度交互的智能体仍不确定。最后，可重复性是一个潜在关切点：代理智能体配置的差异、生成的补全数量或评判模型的选择可能导致不同的推理重构结果，从而影响跨实验的一致性。通过明确承认这些效度威胁，我们旨在提供一个透明的评估，说明在哪些上下文中可以自信地解读沃森的结果。

六、结论

在本文中，我们引入了认知可观测性的概念，这是由基础模型驱动的智能体软件在可观测性方面的一项关键进展。由于大语言模型不透明且非确定性的推理过程，传统的可观测性技术在这些系统中力有不逮。为了应对这一挑战，我们提出并实现了沃森，这是一个新颖的框架，能够在保持标准大语言模型效率的同时，以高保真度回顾性地恢复隐式推理轨迹。

我们的评估表明，沃森能够同时支持手动调试和自动化运行时纠错，在不影响智能体行为或输出的情况下，有效地揭示了先前无法获取的智能体决策背后的“为什么”。通过在真实世界智能体任务上的经验验证，包括MMLU，以及在SWE-bench-lite基准测试上的AutoCodeRover和OpenHands，沃森促进了推理透明度的提升，并支持了智能体软件可靠性和可控性的增强。总体而言，沃森为应对由大语言模型驱动的智能体的可观测性挑战迈出了实用的一步，帮助开发人员更好地解释和改进智能体行为。未来的工作应探索跨智能体类型和任务的更广泛泛化、利用推理轨迹进行干预或再训练的更复杂方法，以及在生产环境中的实时可观测性。随着大语言模型在智能体软件中承担越来越复杂的角色，推进认知可观测性工具对于支持其安全有效的部署至关重要。

REFERENCES

[1] Anthropic Safety Team. *Reasoning models don’t always say what they think*. Anthropic Technical Report. Available at <https://www.anthropic.com/research/reasoning-models-dont-say-think>. 2025.

[2] Arize. *Phoenix by Arize*. last accessed: 2024-10-08. 2024. URL: <https://phoenix.arize.com/>.

[3] Fazl Barez et al. *Chain-of-Thought Is Not Explainability*. 2025. URL: <https://www.alphaxiv.org/abs/2025.02v3>.

[4] Mohammad Bavarian et al. *Efficient Training of Language Models to Fill in the Middle*. 2022. URL: <https://arxiv.org/abs/2207.14255>.

[5] Jiayi Geng Carlos E. Jimenez John Yang. *SWE-bench Lite: A Canonical Subset for Efficient Evaluation of Language Models as Software Engineers*. last accessed: 2024-10-08. 2024. URL: <https://www.swebench.com/lite.html>.

[6] Yupeng Chang et al. “A Survey on Evaluation of Large Language Models”. In: *ACM Transactions on Intelligent Systems and Technology* 15.3 (2024). ISSN: 2157-6904. DOI: 10.1145/3641289. URL: <https://doi.org/10.1145/3641289>.

[7] Boyuan Chen et al. “Towards training reproducible deep learning models”. In: *Proceedings of the 44th International Conference on Software Engineering*. 2022, 2202–2214. ISBN: 9781450392211. DOI: 10.1145/3510003.3510163. URL: <https://doi.org/10.1145/3510003.3510163>.

[8] Yanda Chen et al. *Reasoning Models Don’t Always Say What They Think*. 2025. arXiv: 2505.05410 [cs.CL]. URL: <https://arxiv.org/abs/2505.05410>.

[9] DeepSeek-AI et al. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. 2025. DOI: 10.48550/ARXIV.2501.12948. URL: <https://arxiv.org/abs/2501.12948>.

[10] Django. *django Web Framework*. last accessed: 2024-10-08. 2024. URL: <https://www.djangoproject.com/>.

[11] Liming Dong, Qinghua Lu, and Liming Zhu. *AgentOps: Enabling Observability of LLM Agents*. 2024. DOI: 10.48550/ARXIV.2411.05285. URL: <https://arxiv.org/abs/2411.05285>.

[12] Ximing Dong et al. *PromptExp: Multi-granularity Prompt Explanation of Large Language Models*. under review, preprint available on arXiv. 2024.

[13] Dynatrace. *Dynatrace*. last accessed: 2024-10-08. 2024. URL: <https://www.dynatrace.com/>.

[14] Daniel Fried et al. “InCoder: A Generative Model for Code Infilling and Synthesis”. In: *Proceedings of the 11th International Conference on Learning Representations*. 2023.

[15] *GDB: The GNU Project Debugger*. <https://sourceware.org/gdb/>. Accessed: 2025-05-26.

[16] GitHub. *GitHub Copilot*. last accessed: 2024-10-09. 2024. URL: <https://github.com/features/copilot>.

[17] Siyuan Guo et al. “DS-Agent: Automated Data Science by Empowering Large Language Models with Case-Based Reasoning”. In: *Proceedings of the 41st International Conference on Machine Learning*. 2024. URL: <https://proceedings.mlr.press/v235/guo24b.html>.

[18] *Haptik, Drive Business Efficiency at Scale with Generative AI*. last accessed: 2024-10-08. 2024. URL: <https://www.haptik.ai/>.

[19] Ahmed E. Hassan et al. “Rethinking Software Engineering in the Era of Foundation Models: A Curated Catalogue of Challenges in the Development of Trustworthy FMware”. In: *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. 2024. ISBN: 9798400706585. DOI: 10.1145/3663529.3663849. URL: <https://doi.org/10.1145/3663529.3663849>.

[20] Ahmed E. Hassan et al. *Towards AI-Native Software Engineering (SE 3.0): A Vision and a Challenge Roadmap*. 2024. URL: <https://arxiv.org/abs/2410.06107>.

[21] Helicone. *Helicone*. last accessed: 2024-10-08. 2024. URL: <https://www.helicone.ai/>.

[22] Dan Hendrycks et al. “Measuring Massive Multitask Language Understanding”. In: *International Conference on Learning Representations*. 2021.

[23] Shengran Hu, Cong Lu, and Jeff Clune. *Automated Design of Agentic Systems*. 2024. URL: <https://arxiv.org/abs/2408.08435>.

[24] Lei Huang et al. *A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions*. 2023. URL: <https://arxiv.org/abs/2311.05232>.

[25] Humanloop. *Humanloop*. last accessed: 2024-10-08. 2024. URL: <https://humanloop.com/>.

[26] *Introducing Devin, the first AI software engineer*. last accessed: 2024-10-08. 2024. URL: <https://www.cognition.ai/blog/introducing-devin>.

[27] M. Izadi et al. “Language Models for Code Completion: A Practical Evaluation”. In: *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering*. 2024. DOI: 10.1145/3597503.3639138. URL: <https://doi.ieeecomputersociety.org/10.1145/3597503.3639138>.

[28] Carlos E. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* 2024. URL: <https://arxiv.org/abs/2310.06770>.

[29] LangChain. *LangSmith by LangChain*. last accessed: 2024-10-08. 2024. URL: <https://www.langchain.com/langsmith/>.

[30] Langfuse. *Langfuse*. last accessed: 2024-10-08. 2024. URL: <https://langfuse.com/>.

[31] B. Li et al. “Enjoy your observability: an industrial survey of microservice tracing and analysis”. In: *Empirical Software Engineering* 27.1 (2022). DOI: 10.1007/s10664-021-10063-9.

[32] Xiao-Hui Li et al. “A Survey of Data-Driven and Knowledge-Aware eXplainable AI”. In: *IEEE Transac-*

参考文献

[1] Anthropic 安全团队. Reasoning models don’t always say what they think. Anthropic 技术报告. Available at <https://www.anthropic.com/research/reasoning-models-dont-say-think>. 2025.[2] Arize. Arize Phoenix. last accessed: 2024-10-08. 2024. URL: <https://phoenix.arize.com/>.[3] Fazl Barez et al. Chain-of-Thought Is Not Explainability. 2025. URL: <https://www.alphaxiv.org/abs/2025.02v3>.[4] Mohammad Bavarian et al. Efficient Training of Language Models to Fill in the Middle. 2022. URL: <https://arxiv.org/abs/2207.14255>.[5] Jiayi Geng 卡洛斯·E·希门尼斯 John Yang. SWE-bench Lite: A Canonical Subset for Efficient Evaluation of Language Models as Software Engineers. last accessed: 2024-10-08. 2024. URL: <https://www.swebench.com/lite.html>.[6] Yupeng Chang et al. “A Survey on Evaluation of Large Language Models”. In: ACM 智能系统与技术汇刊 15.3 (2024). ISSN: 2157-6904. DOI: 10.1145/3641289. URL: <https://doi.org/10.1145/3641289>.[7] Boyuan Chen et al. “Towards training reproducible deep learning models”. In: Proceedings of the 44th International Conference on Software Engineering. 2022, 2202–2214. ISBN: 9781450392211. DOI: 10.1145/3510003.3510163. URL: <https://doi.org/10.1145/3510003.3510163>.[8] Yanda Chen et al. Reasoning Models Don’t Always Say What They Think. 2025. arXiv: 2505.05410 [计算语言学]. URL: <https://arxiv.org/abs/2505.05410>.[9] DeepSeek-AI et al. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. 2025. DOI: 10.48550/ARXIV.2501.12948. URL: <https://arxiv.org/abs/2501.12948>.[10] Django. Django 框架. last accessed: 2024-10-08. 2024. URL: <https://www.djangoproject.com/>.[11] Liming Dong, Qinghua Lu, and Liming Zhu. AgentOps: Enabling Observability of LLM Agents. 2024. DOI: 10.48550/ARXIV.2411.05285. URL: <https://arxiv.org/abs/2411.05285>.[12] Ximing Dong et al. PromptExp: Multi-granularity Prompt Explanation of Large Language Models. under review, preprint available on arXiv. 2024.[13] Dynatrace. Dynatrace. last accessed: 2024-10-08. 2024. URL: <https://www.dynatrace.com/>.[14] Daniel Fried et al. “InCoder: A Generative Model for Code Infilling and Synthesis”. In: Proceedings of the 11th International Conference on Learning Representations. 2023.[15] GDB: The GNU Project Debugger. <https://sourceware.org/gdb/>. Accessed: 2025-05-26.[16] GitHub. GitHub Copilot. last accessed: 2024-10-09. 2024. URL: <https://github.com/features/copilot>.

[17] Siyuan Guo等人。“DS-Agent：通过赋予大语言模型基于案例的推理能力实现自动化数据科学”。载于：第41届国际机器学习会议论文集。2024。网址：<https://proceedings.mlr.press/v235/guo24b.html>.[18] Haptik，利用生成式AI规模化提升业务效率。最后访问日期：2024-10-08。2024。网址：<https://www.haptik.ai/>.[19] Ahmed E. Hassan等人。“重新思考基础模型时代的软件工程：构建可信FMware的开发挑战精选目录”。载于：第32届ACM软件工程基础国际会议伴随论文集。2024。ISBN：9798400706585。DOI：10.1145/3663529.3663849。网址：<https://doi.org/10.1145/3663529.3663849>.[20] Ahmed E. Hassan等人。迈向AI原生软件工程（SE 3.0）：愿景与挑战路线图。2024。网址：<https://arxiv.org/abs/2410.06107>.[21] Helicone. Helicone。最后访问日期：2024-10-08。2024。网址：<https://www.helicone.ai/>.[22] Dan Hendrycks等人。“衡量大规模多任务语言理解”。载于：国际学习表征大会。2021。[23] Shengran Hu, Lu Cong，以及杰夫·克鲁恩。智能体系统的自动化设计。2024。网址：<https://arxiv.org/abs/2408.08435>.[24] Huang Lei等人。大语言模型中幻觉的调查：原理、分类、挑战与开放问题。2023。网址：<https://arxiv.org/abs/2311.05232>.[25] Humanloop。Humanloop。最后访问日期：2024-10-08。2024。网址：<https://humanloop.com/>.[26] 介绍Devin，首位AI软件工程师。最后访问日期：2024-10-08。2024。网址：<https://www.cognition.ai/blog/introducing-devin>.[27] M. Izadi等人。“用于代码补全的语言模型：一项实用评估”。载于：第46届IEEE/ACM软件工程国际会议论文集。2024。DOI：10.1145/3597503.3639138。网址：<https://doi.ieeecomputersociety.org/10.1145/3597503.3639138>.[28] 卡洛斯·E·希门尼斯等人。SWE-bench：语言模型能解决真实的GitHub问题吗？2024。网址：<https://arxiv.org/abs/2310.06770>.[29] LangChain。LangChain推出的LangSmith。最后访问日期：2024-10-08。2024。网址：<https://www.langchain.com/langsmith/>.[30] Langfuse。Langfuse。最后访问日期：2024-10-08。2024。网址：<https://langfuse.com/>.[31] B. Li等人。“享受你的可观测性：微服务跟踪与分析的工业调查”。载于：Empirical Software Engineering 27.1 (2022)。DOI：10.1007/s10664-021-10063-9.[32] 李小慧等人。“数据驱动与知识感知的可解释AI调查”。载于：IEEE Transac-

tions on Knowledge and Data Engineering 34.1 (2022), pp. 29–49. DOI: 10.1109/TKDE.2020.2983930.

[33] Q. Vera Liao and Jennifer Wortman Vaughan. *AI Transparency in the Age of LLMs: A Human-Centered Research Roadmap*. 2023. arXiv: 2306.01941 [cs.HC]. URL: <https://arxiv.org/abs/2306.01941>.

[34] Ryan Liu et al. *Mind Your Step (by Step): Chain-of-Thought can Reduce Performance on Tasks where Thinking Makes Humans Worse*. 2024. arXiv: 2410.21333 [cs.LG]. URL: <https://arxiv.org/abs/2410.21333>.

[35] Lunary. *Lunary*. last accessed: 2024-10-08. 2024. URL: <https://lunary.ai/>.

[36] Tula Masterman et al. *The Landscape of Emerging AI Agent Architectures for Reasoning, Planning, and Tool Calling: A Survey*. 2024. URL: <https://arxiv.org/abs/2404.11584>.

[37] Nebuly. *Nebuly - Explicit and Implicit LLM User Feedback Quickguide*. last accessed: 2024-10-08. 2024. URL: <https://www.nebuly.com/blog/explicit-implicit-llm-user-feedback-quickguide>.

[38] OpenAI. *Introducing OpenAI o1*. last accessed: 2024-10-08. 2024. URL: <https://openai.com/o1/>.

[39] Qwak. *Qwak - LLMops*. last accessed: 2024-10-08. 2024. URL: <https://www.qwak.com/products/llmops>.

[40] Noah Shinn et al. “Reflexion: language agents with verbal reinforcement learning”. In: *Proceedings of the 37th International Conference on Neural Information Processing Systems*. NIPS ’23. New Orleans, LA, USA: Curran Associates Inc., 2023.

[41] Benjamin H. Sigelman et al. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Tech. rep. Google, Inc., 2010. URL: <http://research.google.com/archive/papers/dapper-2010-1.pdf>.

[42] DiJia Su et al. *Dualformer: Controllable Fast and Slow Thinking by Learning with Randomized Reasoning Traces*. 2025. arXiv: 2410.09918 [cs.AI]. URL: <https://arxiv.org/abs/2410.09918>.

[43] Traceloop. *Traceloop*. last accessed: 2024-10-08. 2024. URL: <https://www.traceloop.com/>.

[44] Xingyao Wang et al. *OpenHands: An Open Platform for AI Software Developers as Generalist Agents*. 2025. arXiv: 2407.16741 [cs.SE]. URL: <https://arxiv.org/abs/2407.16741>.

[45] Xuezhi Wang et al. *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. 2023. arXiv: 2203.11171 [cs.CL]. URL: <https://arxiv.org/abs/2203.11171>.

[46] Jason Wei et al. “Chain-of-thought prompting elicits reasoning in large language models”. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. 2022. ISBN: 9781713871088.

[47] Weights and Biases. *Weights and Biases - Weave*. last accessed: 2024-10-08. 2024. URL: <https://wandb.ai/site/weave/>.

[48] WhyLabs. *WhyLabs*. last accessed: 2024-10-08. 2024. URL: <https://whylabs.ai/>.

[49] Fangzhi Xu et al. *Are Large Language Models Really Good Logical Reasoners? A Comprehensive Evaluation and Beyond*. 2024. URL: <https://arxiv.org/abs/2306.09841>.

[50] Shunyu Yao et al. “Tree of thoughts: deliberate problem solving with large language models”. In: *Proceedings of the 37th International Conference on Neural Information Processing Systems*. 2024.

[51] Yuntong Zhang et al. “AutoCodeRover: Autonomous Program Improvement”. In: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. 2024. ISBN: 9798400706127. DOI: 10.1145/3650212.3680384. URL: <https://doi.org/10.1145/3650212.3680384>.

tions on 知识与数据工程 34.1 (2022), pp. 29–49. DOI: 10.1109/TKDE.2020.2983930.[33] Q. Vera Liao和Jennifer Wortman Vaughan。大语言模型时代的AI透明性：以人为中心的研发路线图。2023。arXiv: 2306.01941 [cs.HC]。网址：<https://arxiv.org/abs/2306.01941>.[34] Ryan Liu等人。小心你的步子（一步步地）：思维链可能降低在思考会让人变差的任务上的性能。2024。arXiv: 2410.21333 [cs.LG]。网址：<https://arxiv.org/abs/2410.21333>.[35] Lunary. Lunary。最后访问日期：2024-10-08。2024。网址：<https://lunary.ai/>.[36] Tula Masterman等人。用于推理、规划和工具调用的新兴AI智能体架构全景：一项调查。2024。网址：<https://arxiv.org/abs/2404.11584>.[37] Nebuly. Nebuly - 显式与隐式大语言模型用户反馈快速指南。最后访问日期：2024-10-08。2024。网址：[nebuly.com/blog/explicit-implicit-llm-user-feedback-quickguide](https://www.nebuly.com/blog/explicit-implicit-llm-user-feedback-quickguide)。

[38] OpenAI. 介绍OpenAI o1。最后访问日期：2024-10-08。2024。网址：<https://openai.com/o1/>.[39] Qwak. Qwak - LLMops。最后访问日期：2024-10-08。2024。网址：<https://www.qwak.com/products/llmops>.[40] Noah Shinn等人。“Reflexion：具有语言强化学习的语言智能体”。载于：第37届神经信息处理系统国际会议论文集。NIPS ’23。美国路易斯安那州新奥尔良：Curran Associates Inc., 2023。[41] Benjamin H. Sigelman等人。Dapper：大规模分布式系统跟踪基础设施。技术报告。Google, Inc., 2010。网址：<http://research.google.com/archive/papers/dapper-2010-1.pdf>.[42] DiJia Su等人。Dualformer：通过随机推理轨迹学习实现可控的快速和慢速思考。2025。arXiv: 2410.09918 [cs.AI]。网址：<https://arxiv.org/abs/2410.09918>.[43] Traceloop. Traceloop。最后访问日期：2024-10-08。2024。网址：<https://www.traceloop.com/>.[44] Xingyao Wang等人。OpenHands：作为通用智能体的AI软件开发者开放平台。2025。arXiv: 2407.16741 [cs.SE]。网址：<https://arxiv.org/abs/2407.16741>.[45] Xuezhi Wang等人。自一致性改进语言模型中的思维链推理。2023。arXiv: 2203.11171 [cs.CL]。网址：<https://arxiv.org/abs/2203.11171>.[46] Jason Wei等人。“思维链提示在大型语言模型中引出推理”。载于：第36届神经信息处理系统国际会议论文集。2022。ISBN: 9781713871088.[47] Weights and Biases. Weights and Biases - Weave。最后访问日期：2024-10-08。2024。网址：<https://wandb.ai/site/weave/>。

[48] WhyLabs. WhyLabs。最后访问日期：2024-10-08。2024。网址: <https://whylabs.ai/>.[49] 徐方志 等. Are Large Language Models Really Good Logical Reasoners? A Comprehensive Evaluation and Beyond. 2024. 网址: <https://arxiv.org/abs/2306.09841>.[50] 姚顺雨 等. “思维树：基于大语言模型的深思熟虑的问题解决”. In: Proceedings of the 37th International Conference on Neural Information Processing Systems. 2024.[51] 张云通 等. “AutoCodeRover: 自主程序改进”. In: 第33届ACM SIGSOFT软件测试与分析国际研讨会论文集. 2024. ISBN: 9798400706127. DOI: 10.1145/3650212.3680384. 网址: <https://doi.org/10.1145/3650212.3680384>.